Chương 5: HỆ LUẬT SINH
Production system

- Tìm kiếm đệ qui
- Hệ luật sinh: Định nghĩa và ứng dụng
- Tìm kiếm trên hệ luật sinh

Phạm Minh Tuấn

Đặc tính dữ liệu và điều khiển của SSS

- Các đặc tính của giải thuật SSS:
  - Lời giải là một PATH từ điểm START đến điểm GOAL
  - Tìm kiếm là sự kiểm tra có hệ thống các đường dẫn đến GOAL
  - Backtracking cho phép giải thuật "phục hồi" khi đi vào một nhánh không có đáp án.
  - Các danh sách sẽ giữ các trạng thái đang xem xét:
    - Danh sách Open: cho phép hệ thống backtrack về các trạng thái chưa được xét.
    - Danh sách Close: cho phép hệ thống kiểm tra sự quay vòng tránh lặp vô tận
  - Dùng STACK cho DFS, QUEUE cho BFS và dùng PRIORITY QUEUE cho BFS.

Tìm kiếm đệ qui – Tại sao???

- Tìm kiếm đệ qui là giải thuật tìm kiếm trên SSS với các đặc tính:
  - Ngắn gọn xúc tích hơn
  - Tiếp cận của giải thuật tự nhiên hơn
  - Hợp nhất với phương thức hiện thực của Logic vị từ
- Các giải thuật tìm kiếm đệ qui chính:
  - Tìm kiếm đệ qui – Recursive Search (RS)
  - Tìm kiếm theo mẫu – Pattern Directed Search (PDS)
- Các giải thuật tìm kiếm đệ qui được sử dụng rộng rải trong các shell của các Hệ chuyên gia (Expert System).
- Pattern Directed Search là nền tản của PROLOG

Thế nào là đệ qui?

- Đệ qui là sự định nghĩa một đối tượng bằng cách sử dụng chính đối tượng đó – Toán học
- Đệ qui được dùng để định nghĩa và phân tích các cấu trúc dữ liệu cũng như các thủ tục xử lý trong ngành máy tính.
- Một thủ tục đệ qui bao gồm:
  - Thành phần đệ qui, trong đó thủ tục gọi chính nó để lặp lại chuỗi các thao tác.
  - Thành phần dừng dùng để dừng quá trình đệ qui vô tận. (lặp vô tận)
- Hai thành phần này tồn tại đồng thời trong tất cả các định nghĩa đệ qui cũng như giải thuật đệ qui.
- Đệ qui là một cấu trúc điều khiển dữ liệu tự nhiên cho những cấu trúc không xác định số phần tử cố định: list, tree, và đồ thị.

Thủ tục đệ qui – ví dụ

```
Function Member(item, list);
 begin
  if List rỗng then return (Fail)
  else
   if Item = phần tử đầu của list then return
      (succes)
     else
      begin
       Tail:= List \ phần tử đầu;
        member (item, Tail);
      end
 end;
```

- Đệ qui có đầy đủ tính năng của các cấu trúc điều khiển truyền thống như Loop và rẽ nhánh➔mọi chương trình viết được bằng cấu trúc truyền thống đều có thể viết đệ qui.
- Đệ qui thích hợp biểu diễn các cấu trúc toán học ➔ thuận tiện trong việc kiểm tra tính đúng đắn của giải thuật đệ qui.
- Công thức đệ qui cũng thường được dùng trong việc sinh và kiểm tra chương trình tự động.
- Đệ qui là công cụ tự nhiên và mạnh mẻ cho hiện thực các chiến lược giải quyết vấn đề của AI.

Tại sao?

Giải thuật DFS đệ qui - DFS

```
Function Depth_First_Search;
Begin
 if Open rỗng then return (fail);
Current_state := phần tử đầu tiên của open;
If (current_state là mục tiêu) then return (Success)
else begin
  open:=phần đuôi của open;
 Closed := Closed + current_state;
 for mỗi phần tử con Y của current_state do
  if not (Y  in close) and not (Y in open) then thêm Y vào đầu của Open;
End;
depth_first_search;
End;
```

Nhược điểm của recursive ?

Pattern-Directed Search (PDS)

- Các giải thuật Search đã tìm hiểu và Recursive Seach không trình bày cách biểu diễn một trạng thái trong không gian trạng thái cũng như cách sinh các trạng thái mới.
- Pattern-Directed Search là một giải thuật search đệ quy dùng Logic Vị từ để hiện thực việc sinh các trạng thái mới.
- Paterm-Directed Search xuất phát từ goal và các modus ponen dạng q(x)->p(x) để chuyển trạng thái. Các modus ponen này gọi là các luật sinh.
- Giải thuật: Xuất phát từ goal P, áp dụng một giải thuật để tìm các rule với P ở vế phải , sau đó xem vế trái Q là subgoal. Đệ quy với Q cho đến khi Qx là một sự kiện trong kho tri thức.

Sự kiện (FACT) trong kho tri thức?????

Giải thuật PDS

Function Pattern_search(current_goal);
Begin
If current_goal có trong closed then retuen fail
 else thêm current_goal vào trong closed;
 while còn trong database các rule hay fact chưa xét
  begin case
   current_goal trùng với fact:
    return(success);
   current_goal là một phép hội:
    begin
     for mỗi thành phần hội Pi do
      pattern_search(Pi);
If tất cả các hội đều success
 then return success else return fail.
 end;
Current_goal là vế phải một rule:
  begin
    áp dụng các thành phần vào vế trái Q.
    if pattern_search(Q) then return success else return fail;
 end;
end; /* case
Return fail;
End;

Giải thuật PDS

- PDS dùng các rule và thành phần hội để sinh các trạng thái con.
- Tách bạch quá trình điều khiển của giải thuật và dữ liệu của giải thuật
  - ➔ Cùng giải thuật chỉ cần thay đổi database : Fact & Rule ta sẽ áp dụng cho các bài toán khác nhau.
  - ➔Xây dựng các shell và có thể vận hành cho các hệ thống khác nhau bằng cách thay đổi Database.
- Để đơn giản hoá giải thuật chưa giải quyết ở mức độ có các biến trong các rule. Ví dụ P(x)^Q(x) chỉ thõa khi P và Q cùng thỏa với cùng giá trị X.
- Các phép ¬, v,.. cũng chưa giải quyết trong giải thuật này.

Ví dụ: Bài toán mã đi tuần

Đặc tả bài toán: tìm đường đi cho con mã qua tất cả các ô trên bàn cờ. Ví dụ với bàn cờ 3x3.

move(1,8)	move(1,6)	move(2,9)	move(2,7)
move(3,4)	move(3,8)	move(4,9)	move(4,3)
move(6,1)	move(6,7)	move(7,2)	move(7,6)
move(8,3)	move(8,1)	move(9,2)	move(9,4)

1	2	3
4	5	6
7	8	9

∀X path(X,X);

∀X,Y [path(X,Y) ← ∃Z[move(X,Z]^path(Z,Y)]]

Giải thuật PDS đầy đủ

Function Pattern_search(current_goal);
Begin
If current_goal có trong Closed then return fail else thêm current_goal vào Closed;
while còn các rule hay fact chưa xét
 begin
 case
 current_goal trùng với fact: return(success);
 current_goal là negated(¬p):
  begin
   if pattern_search(p) then return fail
    else return{}
  end;
 current_goal là một phép hội:
   begin
    for mỗi thành phần hội Pi do
    if not (pattern_search(Pi)) the return fail else thay thế tất cả các biến cho các thành phần hội khác.
    If tất cả các hội đều success then return các thành phần hội else return fail.
   end;
 current_goal là phép tuyển:
  begin
   repeat cho mọi thành phần tuyển Vi;
   until (pattern_search(Vi) or (hết thành phần hội)
   if pattern_search (Vi) then return các thay thế else return fail;
 end;
Current_goal là vế phải một rule:
 begin
   áp dụng các thành phần vào vế trái Q.
   if pattern_search(Q) then return kết hợp của Current_goal và các thay thế của Q else return fail;
end; end; /* case*/
return fail; End;

Hệ Luật Sinh – Production System

- **Khái niệm**: Hệ luật sinh là một mô hình tính toán quan trọng trong các bài toán tìm kiếm cũng như mô phỏng cách giải quyết vấn đề của con người trong lĩnh vực ứng dụng AI.
- **Định nghĩa**: Hệ luật sinh là một mô hình tính toán cung cấp cơ chế điều khiển Pattern_directed trong quá trình giải quyết vấn đề (Proplem solving process).

- **Cấu trúc hệ luật sinh bao gồm 3 thành phần**:
  1. Production rules (Tập luật sản sinh)
  2. Working memory (Vùng nhớ làm việc)
  3. Recognize-action control (Bộ điều khiển nhận dạng và hành động)

Định nghĩa (tt) – Production Rule

- **Production rules**: là một tập các luật sản sinh được đặc tả dạng:

  Condition – Action (điều kiện – hành động)

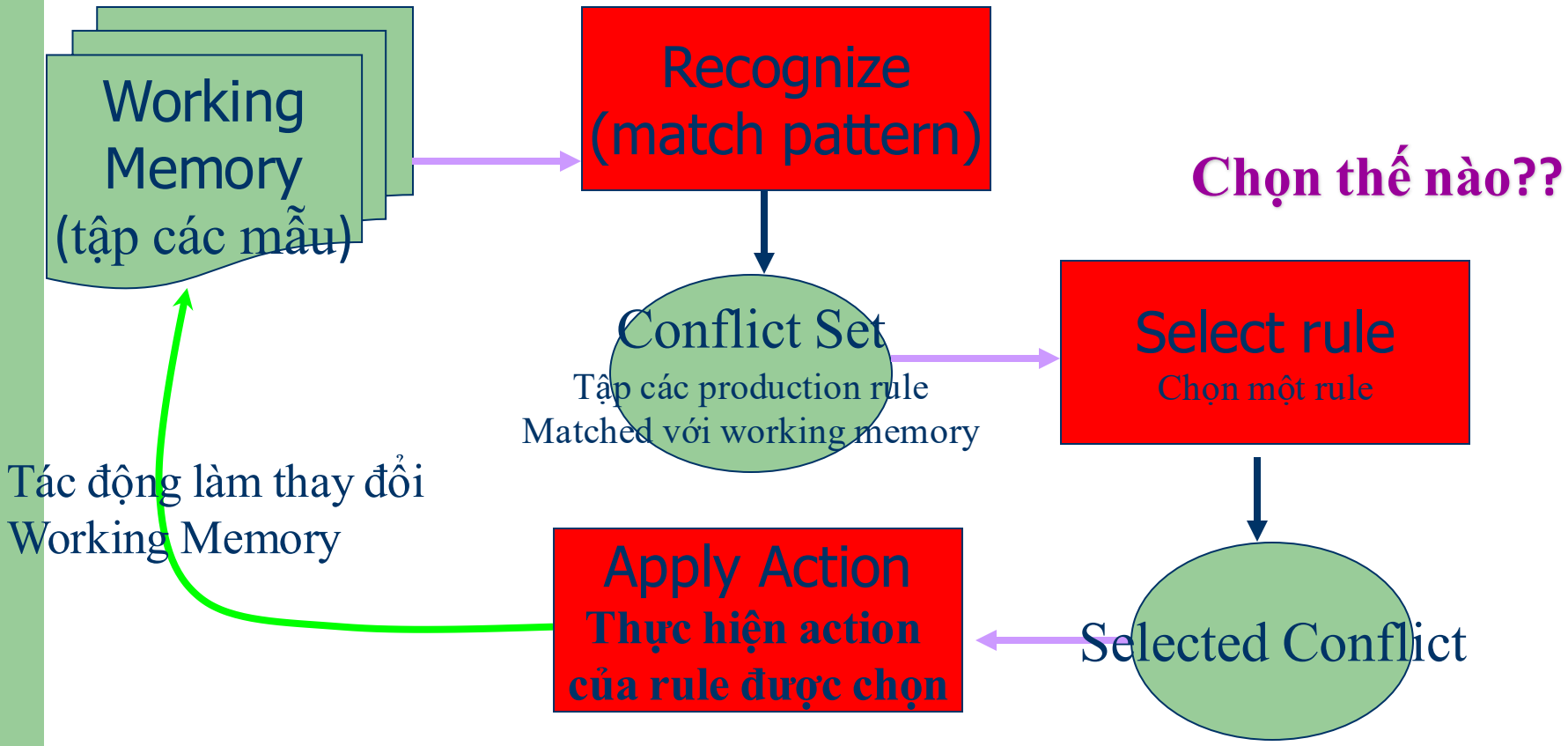
- Một luật là một mắt xích của kho tri thức giải quyết vấn đề. Kho tri thức là một database của các production rules.
- Thành phần Condition: là một mẫu (pattern) dùng xác định điều kiện áp dụng của rule cho một vấn đề tương ứng.
- Thành phần action: mô tả bước giải quyết vấn đề tương ứng sẽ được thực hiện. Đây là phần sẽ tác động lên hiện trạng của không gian tìm kiếm.

Định nghĩa (tt) – Working memory

- **Working memory** chứa những đặc tả trạng thái hiện tại của quá trình suy luận. Chúng được lưu trữ như là tập các mẫu.
- Những đặc tả này là các mẫu để so trùng với các condition của các production rules.
- Khi một production rule được so trùng phần condition thì phần action của nó có thể được áp dụng, và phần action này được xây dựng đặc thù để tác động trực tiếp lên working memory.
- Working memory được khởi tạo bằng trạng thái bắt đầu của vấn đề cần giải quyết.
- Working memory diễn tả hiện trạng của vấn đề cần suy luận

Định nghĩa (tt) – Recognize-action

- Đây là cấu trúc điều khiển dùng trong production system. Quy trình hoạt động của Recognize -Action



Các vấn đề khác

- Chọn lựa Conflict để thực hiện

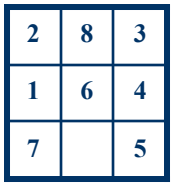
  Có thể chọn bằng cách đơn giản hay áp dụng các chiến lược lựa chọn heurictic → Ứng dụng meta knowledge.

Nếu gọi chiến lược heuristic chọn conflict là meta knowledge thì knowledge "thường" ở đâu trong hệ thống?

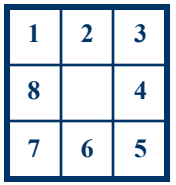
- Các hệ luật sinh đơn thuần không cung cấp cơ chế để quay lui khi việc áp dụng các action làm cho working memory thay đổi dẫn đến lúc không còn production rule nào có thể áp dụng được→ Hệ thống sẽ dừng.
- Cần cung cấp cơ chế backtracking (Thường dùng UNDO). Tuy nhiên cần chú ý để tránh lặp vòng.

Ví dụ: bài toán 8 Puzzle

- **Working Memory**



Current state



Goal state

- **Production Rules**

Goal state	→ Halt
Blank is not on top edge	→ Move the blank up
Blank is not on the right edge	→ Move the blank right
Blank is not on the bottom edge	→ Move the blank down
Blank is not on the left edge	→ Move the left down

- **Recognize-Action**

1- Try each production rule in order

2- Do not allow loops

3- Stop when goal is found

Điều khiển tìm kiếm trong hệ luật sinh

- Chiến lược Data-Driven / Goal-Driven:
  - Data-Driven: bắt đầu với problem trong working memory, matching các condition trong production rule → conflict → áp dụng các action → thay đổi working memory. Lặp lại cho đến khi đạt được goal state.
  - Goal-Driven: Bắt đầu với mô tả goal trong working memory, matching với các kết quả của Action → sinh tập các condition→ đưa các condition vào trong working memory. Lặp lại cho đến khi working memory chứa FACT.
- Điều khiển qua cấu trúc rule: Dùng các biến đổi tương đương của các biểu thức trong rule để điều khiển quá trình tìm kiếm.
- Điều khiển bằng sự phân tích các Conflict: Các conflict có thể được chọn lựa thông qua các Heuristic. áp dụng các meta knowledge trong việc chọn conflict. Ví dụ:

1) Refraction: **Khi một rule đã được áp dụng, nó sẽ không được dùng nữa cho đến khi thánh phần trùng lặp với rule cũ trong working memory được thay đỗi.**
2) Recency: **Chọn rule có phần condition match với phần mới thêm vào working memory. Theo đuổi một hướng triển khai.**
3) Specificity: **Rule nào càng được đặc tả chi tiết càng được ưu tiên cao.**

Các ưu điểm của Hệ luật sinh

- Hệ luật sinh là khung làm việc tổng quát để thực thi các giải thuật tìm kiếm. Với đặc tính đơn giản, dễ sửa đổi, và linh động, hệ luật sinh được dùng như một công cụ quan trọng để xây dựng các hệ chuyên gia và các ứng dụng AI khác
- Các ưu điểm của Hệ luật sinh:
  - Tách bạch giữa Tri thức & Điều khiển:
    - Điều khiển: nằm trong chu trình Recognize-Action
    - Tri thức: được chứa đựng trong bản thân các luật sinh.
    - ➔ Cung cấp khả năng cập nhật tri thức mà không cần điều chỉnh chương trình. Thay đổi mã chương trình mà không ảnh hưởng đến tập luật sinh.
  - Dễ dàng áp dụng trong tìm kiếm trên không gian trạng thái. Các state của working memory là các node. Các production rule là các chuyển đổi giữa các trạng thái (cơ chế sinh các trạng thái mới)
  - Tính độc lập của các luật sinh.
  - Khả năng áp dụng heuristic cho việc điều khiển quá trình hoạt động.
  - Theo dõi và giải thích quá trình hoạt động
  - Độc lập với ngôn ngữ & có thể dùng như kỹ thuật mô phỏng giải pháp của người.

Chương 6: HỆ CHUYÊN GIA (ES)

- Giới thiệu về hệ chuyên gia
- Mô hình hệ chuyên gia: dự trên luật, dựa trên frame
- Phát triển một hệ chuyên gia

Phạm Minh Tuấn

Nội dung.

- Giới thiệu về hệ chuyên gia.
  - Định nghĩa, khả năng ứng dụng.
  - Cấu trúc, các đặc trưng cơ bản của ES.
  - Biểu diễn tri thức.
  - Các kỹ thuật suy luận.
- Khảo sát một vài hệ chuyên gia đã có.
  - XCON: ES trợ giúp cấu hình hệ thống máy tính của DEC
  - MYCIN: ES chuẩn đoán bệnh nhiễm trùng máu.
- Hệ chuyên gia dựa trên luật.
  - Kiến trúc, thiết kế.
  - Ưu - nhược điểm.
- Hệ chuyên gia dựa vào Frame.
  - Kiến trúc, thiết kế.
  - Ưu - nhược điểm.

Giới thiệu về hệ chuyên gia.

- Định nghĩa:

  Hệ chuyên gia là một chương trình được thiết kế để theo mô hình có khả năng giải quyết vấn đề của chuyên gia con người.

- Sơ đồ khối cơ bản:



Giới thiệu về hệ chuyên gia.

- Cơ sở tri thức:

  ♣ Dùng để chứa tri thức trong một lĩnh vực nào đó, tri thức này do chuyên gia con người chuyển giao.

  ♣ Nó bao gồm: các khái niệm cơ bản, các sự kiện, các luật và quan hệ giữa chúng.

  Ví dụ:

- Tri thức về *bệnh nhiễm trùng máu* do các bác sĩ chuyên khoa này chuyển giao.
- Tri thức về *chiến lược đầu tư* do các nhà cố vấn đầu tư chuyển giao.
- Tri thức về sự diễn dịch dữ liệu khảo sát địa vật lý do các kỹ sư địa chất chuyển giao.
- …?

Giới thiệu về hệ chuyên gia.

- Động cơ suy luận:

  Là *bộ xử lý* cho tri thức, được mô hình sao cho giống với việc suy luận của chuyên gia con người. Bộ xử lý này làm việc dựa trên thông tin mà người dùng mô tả về vấn đề, kết hợp với CSTT, cho ra kết luận hay đề nghị.

- Tạo sao phải xây dựng ES ?

  Chuyên gia con người là tài nguyên quý giá cho nhiều tổ chức. Họ có thể giải quyết những vấn đề khó, hiệu quả,…. Vậy có giá trị không khi chúng ta xây dựng một chương trình có khả năng như chuyên gia con người ?Một số mặt nào đó còn có thể hơn hẳn. Xem bảng so sánh sau:

Giới thiệu về hệ chuyên gia.

♣ Bảng so sánh:

Tiêu chí	CG con người	ES.
1. Sẵng dùng	TG. hành chính Mọi lúc.	
2. Vị trí	Cục bộ	Mọi nơi.
3. An toàn	không thể thay thế	Có thể thay thế.
4. Có thể chết	Có	Không.
5. Hiệu suất	Thay đổi	Hằng số.
6. Tốc độ	Thay đổi	Hằng số (thường nhanh hơn)
7. Chi phí	Cao	Có thể cố gắng.

Giới thiệu về hệ chuyên gia.

- Vài lý do để phát triển ES thay cho chuyên gia con người:
  - Tạo cho tính chuyên gia sẵn dùng ở mọi nơi, mọi lúc.
  - Tự động hoá các công việc đòi hỏi chuyên gia.
  - Các chuyên gia đang nghỉ hưu hay chuyển đến nơi khác – Cần thay thế.
  - Thuê chuyên gia với chi phí quá lớn.
  - Tính chuyên gia cần thiết trong các môi trường làm việc không thân thiện, ở đó hỏi một ES sẽ nhanh hơn một chuyên gia con người.
  - Phát triển ES để trợ giúp cho chuyên gia con người.

Giới thiệu về hệ chuyên gia.

- Các kiểu vấn đề thường được giải quyết bởi ES:

♣ Điều khiển:

♣ Thiết kế:

♣ Chuẩn đoán:

♣ Dạy học:

♣ Diễn dịch:

♣ Giám sát:

♣ Hoạch định:

♣ Dự đoán:

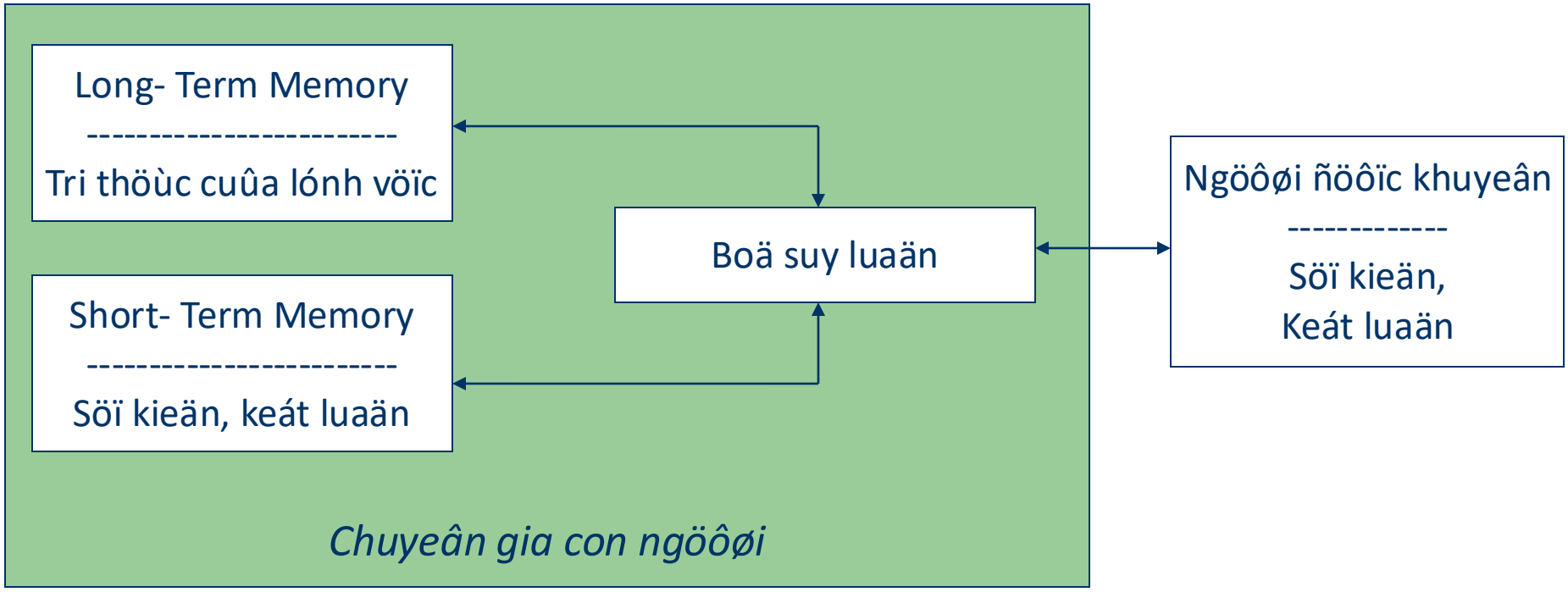
♣ Lựa chọn:

♣ Mô phỏng:

Cấu trúc của ES.

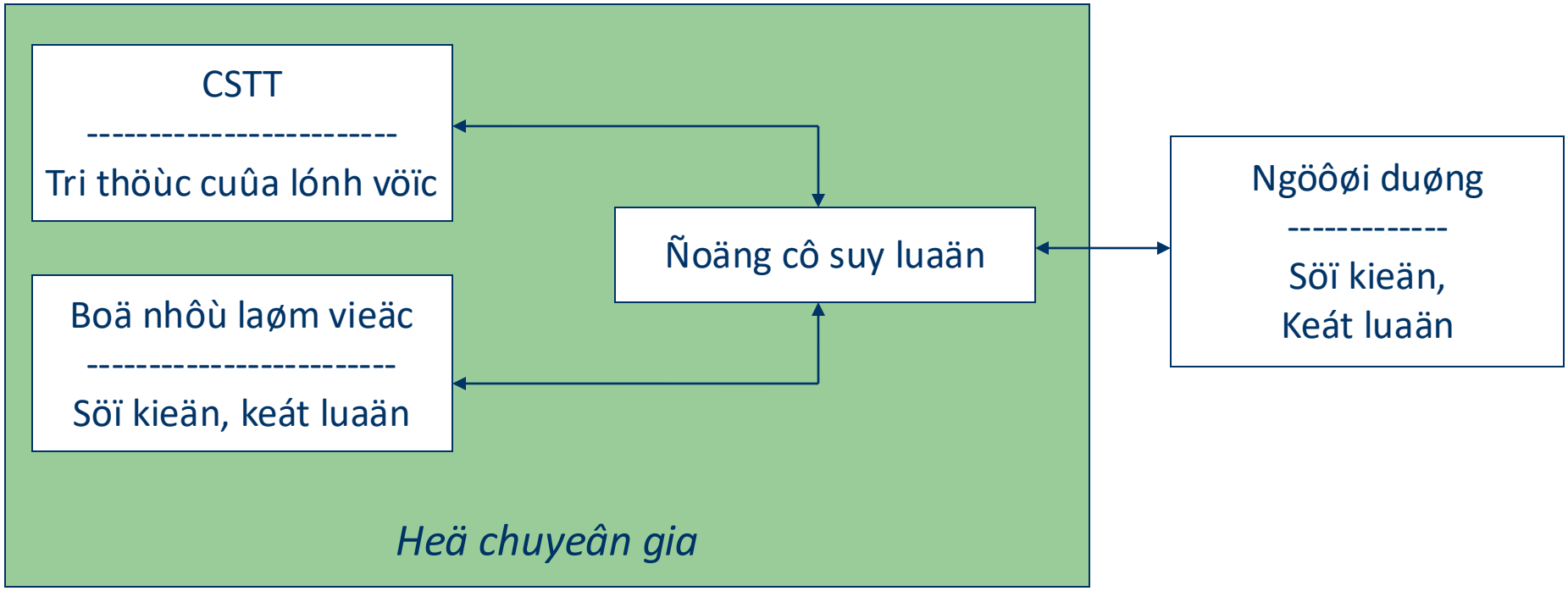
- Cấu trúc của ES:

ES mô phỏng khả năng giải quyết vấn đề của chuyên gia con người. Do vậy, chúng ta cần xem xét cách thức giải quyết của chuyên gia con người, để từ đó mô phỏng.



Cấu trúc của ES.

♣ Cách giải quyết vấn đề ở ES:



Cấu trúc của ES.

- CSTT:

  Là một bộ phận của ES nhằm chứa tri thức của lĩnh vực.

  ♣ ES chứa tri thức của chuyên gia con người trong một bộ phận được gọi là CSTT. Để có tri thức này, người kỹ sư tri thức phải thu thập tri thức từ chuyên gia con người rồi mã hoá vào CSTT – cách thức mã hoá sẽ được đề cập trong phần kỹ thuật biểu diễn tri thức.

  ♣ Một trong các cách tiêu biểu để biểu diễn là dùng luật, như sau:

  RULE 1:

  IF "Xe car không thể khởi động được"

  THEN "Vấn đề trong hệ thống điện"

  RULE 2:

  IF "Vấn đề trong hệ thống điện"

  AND "Điện thế AC-quy nhỏ hơn 10Volt"

  THEN lỗi tại bộ AC-quy"

Cấu trúc của ES.

- Bộ nhớ làm việc:

  Là bộ phận của ES dùng để chứa các sự kiện của vấn đề. Các sự kiện này có thể do người dùng nhập vào lúc đầu hay do ES sinh ra trong quá trình làm việc.

  - Với ES dùng cho nhiều người cùng thì bộ nhớ làm việc thường phân nhóm theo phiên làm việc (session) của người dùng. Đó là trường hợp một ES chung cho nhiều người dùng từ xa.

  - Nhiều ES cũng tận dụng các thông tin được chứa trong các nguồn ngoài như: CSDL, bảng tính, sensor,…ES sẽ tải thông tin này vào bộ nhớ làm việc đầu mỗi session hay khi cần thiết.

Cấu trúc của ES.

- Động cơ suy luận:

Là bộ xử lý trong hệ chuyên gia, là nhiệm vụ *so trùng các sự kiện được chứa trong bộ nhớ làm việc với tri thức được chứa trong CSTT nhằm dẫn ra kết luận cho vấn đề.*

♣ Tiêu biểu, nếu CSTT có chứa luật, ES sẽ tìm ra luật mà các tiên đề của luật so trùng với các sự kiện được chứa trong bộ nhớ làm việc, lúc đó ES sẽ thêm các kết luận của luật đó vào bộ nhớ làm việc, rồi tiếp tục tìm ra sự so trùng khác – giống như nguyên lý hoạt động của hệ luật sinh.

♣ Ví dụ: Giả sử CSTT chỉ với hai luật nêu trên

Bước 1:

ES: Có phải xe car không khởi động được ?

Người dùng: Đúng.

Cấu trúc của ES.

Chú thích: *Người dùng trả lời "Đúng", nên ES thêm vào bộ nhớ làm việc sự kiện để mô tả:*

"Xe car không thể khởi động được"

Động cơ suy diễn của ES làm nhiệm vụ so trùng, nhận thấy RULE 1 có thể so trùng được, nên nó thêm vào bộ nhớ làm việc phần kết luận của RULE 1, đó là:

"Vấn đề trong hệ thống điện"

Bước 2:

ES: Có phải điện Ac-quy dưới 10 Volt?

Người dùng: Đúng.

Chú thích: *Người dùng trả lời "Đúng", nên ES thêm vào bộ nhớ làm việc sự kiện để mô tả:*

"Điện thế Ac-quy nhỏ hơn 10Volt"

Động cơ suy diễn của ES làm nhiệm vụ so trùng, nhận thấy RULE 2 có thể so trùng được, nên nó thêm vào bộ nhớ làm việc phần kết luận của RULE 2, đó là:

lỗi tại bộ Ac-quy" – phiên làm việc cũng kết thúc vì CSTT chỉ gồm hai luật trên.

Cấu trúc của ES.

- Tiện ích giải thích.

Một trong các điểm nổi bật của ES là khả năng giải thích về suy luận của nó. ES còn có một khối cơ bản nữa trong cấu trúc của nó đó là: *khối tiện ích giải thích*. Với khối này ES có thể cung cấp cho người dùng các khả năng giải thích:

- Tại sao ES lại hỏi câu hỏi nào đó. (WHY)
- Bằng cách nào ES có thể suy ra kết luận nào đó. (HOW)

Khối tiện ích giải thích thuận tiện cho cả người phát triển ES và người dùng. Người phát triển có thể nhờ đó khám phá các lỗi trong tri thức của ES. Người thì có thể yên tâm hơn khi nhận một kết luận nào đó, không cần thiết phải quan tâm với cấu trúc tri thức của ES.

♣ Giải thích bằng cách nào (HOW)

Ngoài chức năng cung cấp cho người dùng kết quả suy luận cuối cùng, ES càng có thể cung cấp nó đạt đến kết luận bằng cách nào.

Cấu trúc của ES.

Khả năng giải thích bằng cách nào rất quan trọng với ES, nó làm cho ES khác với các chương trình truyền thống. Các chương trình truyền thống làm việc trên các bài toán được định nghĩa tốt, kết quả của chương trình đã được kiểm nghiệm trước khi chuyển giao. ES làm việc trên các bài toán thiếu nhiều thông tin, thậm chí tri thức còn có thể cập nhật khi ES đã được triển khai – chính vì vậy mà kết luận của nó phải được giải thích bằng cách nào để cho kết quả cuối cùng có giá trị hơn.

Ví dụ: trong phiên làm việc của ES nói trên.

ES: lỗi tại bộ Ac-quy".

Người dùng: HOW

ES: "Bởi vì, khi xe của bạn không khởi động được, tôi cho rằng đã có vấn đề trong hệ thống điện. Mỗi lần, tôi thấy điện thế của ac-quy dưới 10 volt, tôi biết rằng ac-quy đã hư."

- Khả năng giải thích HOW của ES có thể thực hiện được bằng cách cho phép nó theo vết các luật mà nó đã sử dụng vào suy luận.

Cấu trúc của ES.

♣ Giải thích tại sao (WHY)

khả năng của ES cung cấp cách giải thích: "tạo sao nó lại hỏi một câu hỏi nào đó." Với khả năng này, người dùng cũng có thể theo vết được luồng suy luận của ES và yên tâm hơn với kết quả.

Ví dụ: trong phiên làm việc với ES nói trên.

ES: Có phải xe car không khởi động được ?

Người dùng: WHY

ES: "Nếu tôi biết rằng xe car không khởi động được, thì tôi thường cho rằng có vấn đề trong hệ thống điện."

Khi chúng ta hỏi WHY, ES thường đáp trả bằng cách mô tả cái gì mà nó có thể kết luận từ câu trả lời. Hầu hết các ES thường đáp trả bằng cách hiện luật mà nó đang quan tâm.

Cấu trúc của ES.

- Giao diện người dùng:

  Giao diện cũng là một thành phần quan trọng của ES, nó giúp cho ES có thể đặt câu hỏi với người dùng và nhận về câu trả lời chính xác. Yêu cầu cao nhất cho giao diện là có khả năng cung cấp cách hỏi đáp tương tự như giữa người - với - người.

  Khi hiện thực hệ thống, vì những hạn chế của kỹ thuật hiện tại nên người thiết kế phải nghĩ đến những hình thức giao tiếp sao cho tiện lợi, tuy chưa thật giống với "người- người". Cụ thể, có thể dùng giao diện đồ họa , dạng menu chọn, phát âm câu hỏi, … *cũng cần phải tính đến khả năng dùng web như môi trường tương tác.*

Các đặc trưng cơ bản của ES.

- Phân tách tri thức và điều khiển.

  Đã đề cập trong hệ luật sinh. Đây cũng là đặc điểm phân biệt giữa chương trình truyền thống và ES.

  Hãy so sánh khả năng thay đổi tri thức của vấn đề giữa hai loại chương trình nói trên.

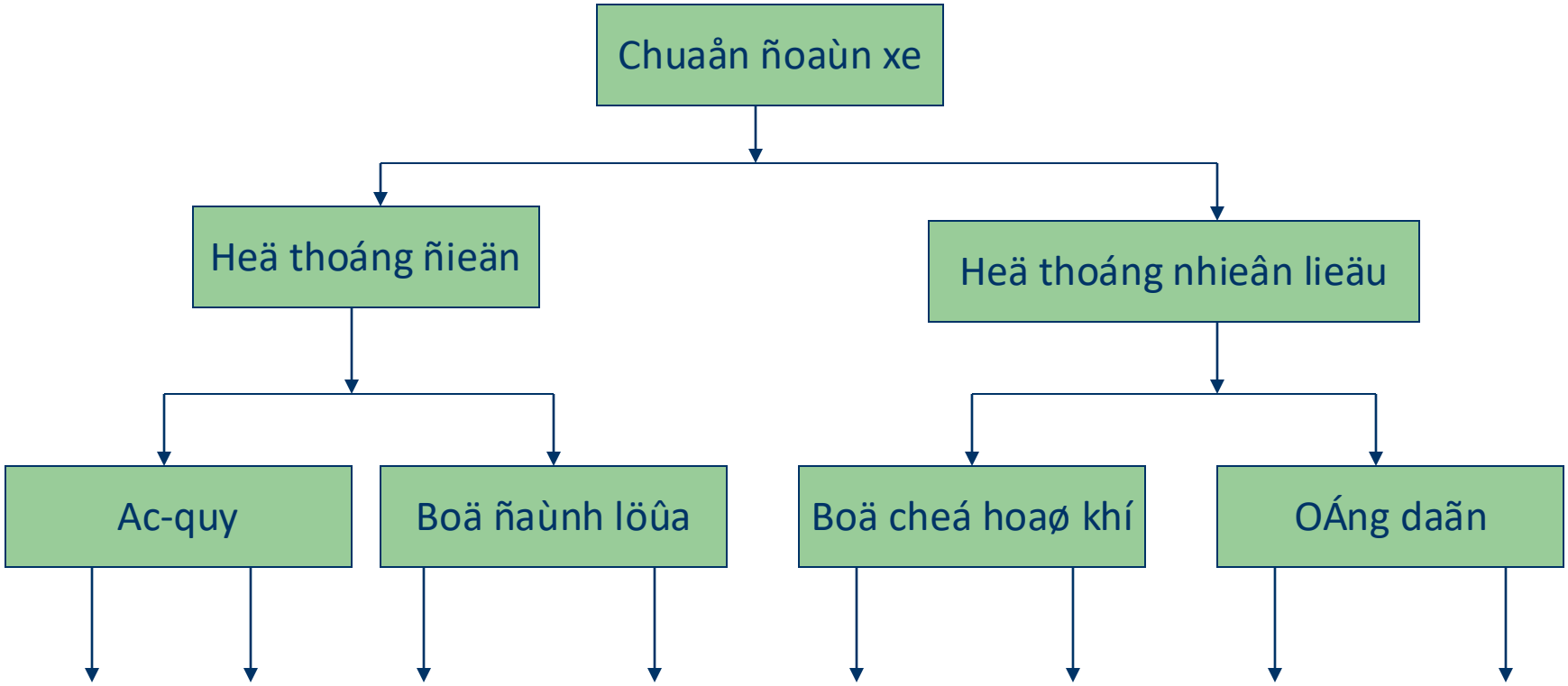
- Sỡ hữu tri thức chuyên gia.

  ES có chứa tri thức của lĩnh vực trong CSTT. Nhờ có tri thức mà nó có giá trị. Đặc biệt là tri thức này có thể được nhân ra thành nhiều bản, có thể cập nhật trong khi hệ thống đã được triển khai.

- Tính chuyên gia trong lĩnh vực hẹp.

  Cũng giống như chuyên gia con người, ES được phát triển nhằm vào một lĩnh vực hẹp. Điều này cũng dễ hiểu, vì lý do: trong lĩnh vực hẹp đó số lượng tri thức cũng nhỏ hơn, và giúp cho người thiết dễ dàng quản lý hơn, dể dàng thử nghiệm chiến lược điều khiển trong động cơ suy diễn. Người thiết thường chia tri thức theo từng mảng như hình sau để quản lý nó.

Các đặc trưng cơ bản của ES.



Các đặc trưng cơ bản của ES.

- Suy luận trên ký hiệu.

  Chúng ta có thể dùng ký hiệu để thể hiện tri thức cho ES. Chính vì vậy mà có thể tận dụng được các giải thuật trên ký hiệu để tri thức thức, như các giải thuật đã đề cập trong chương 2 – phần phép toán vị từ.

- Suy luận có heuristic

  Chuyên gia con người có thể từ kinh nghiệm của mình để dẫn ra cách giải quyết vấn đề hiệu quả hơn, ví dụ:

  Khi chuẩn đoán xe, họ có thể giả thiết cách làm:

  - Luôn luôn kiểm tra luật về hệ thống điện trước các luật khác.

  Hay một bác sĩ chuyên khoa có thể giả thiết:

  - Nếu nghi ngờ bị ung thư, thì kiểm tra dòng họ trước.

  Để có thể hiện thực trong ES, người thiết kế cần phải có cách đánh giá thứ tự ưu tiên của các luật, để từ một ngữ cảnh nào đó có thể chọn một luật có lý nhất để bắt đầu.

Các đặc trưng cơ bản của ES.

- Cho phép suy luận không chính xác.

  ES có một khả năng rất mạnh đó là: nó có thể làm việc với các vấn đề đang thiếu thông tin, hay có nhưng hỗn tạp, không rõ ràng. Cũng giống như trường hợp: một ekip bác sĩ đang phải cứu một bệnh nhân hấp hối, lúc đó họ không còn kịp thời gian để làm tất cả các xét nghiệm cần thiết. Khi thiếu thông tin như vậy họ đành tiến hành những cách có lý nhất theo họ. Chúng ta cũng có thể hiện thực ES có tính chất đó bằng cách đưa vào những luật tương ứng với tình huống thiếu thông tin để động cơ suy diễn vận dụng.

- Bị giới hạn vào vấn đề giải quyết.

  Không phải mọi vấn đề đều có thể giải quyết bởi ES. Cụ thể, nếu lĩnh vực chúng muốn xây dựng ES hiện tại chưa có, chưa cần một chuyên gia con người thì việc xây dựng ES khó mà thành công.

- Giải quyết các vấn đề có độ phức tạp vừa phải.

  Nếu vấn đề quá khó, yêu cầu chuyên gia con người đến vài giờ, cần thiết nghĩ đến khả năng chia thành nhiều bài toán con tương ứng mỗi ES.

Các đặc trưng cơ bản của ES.

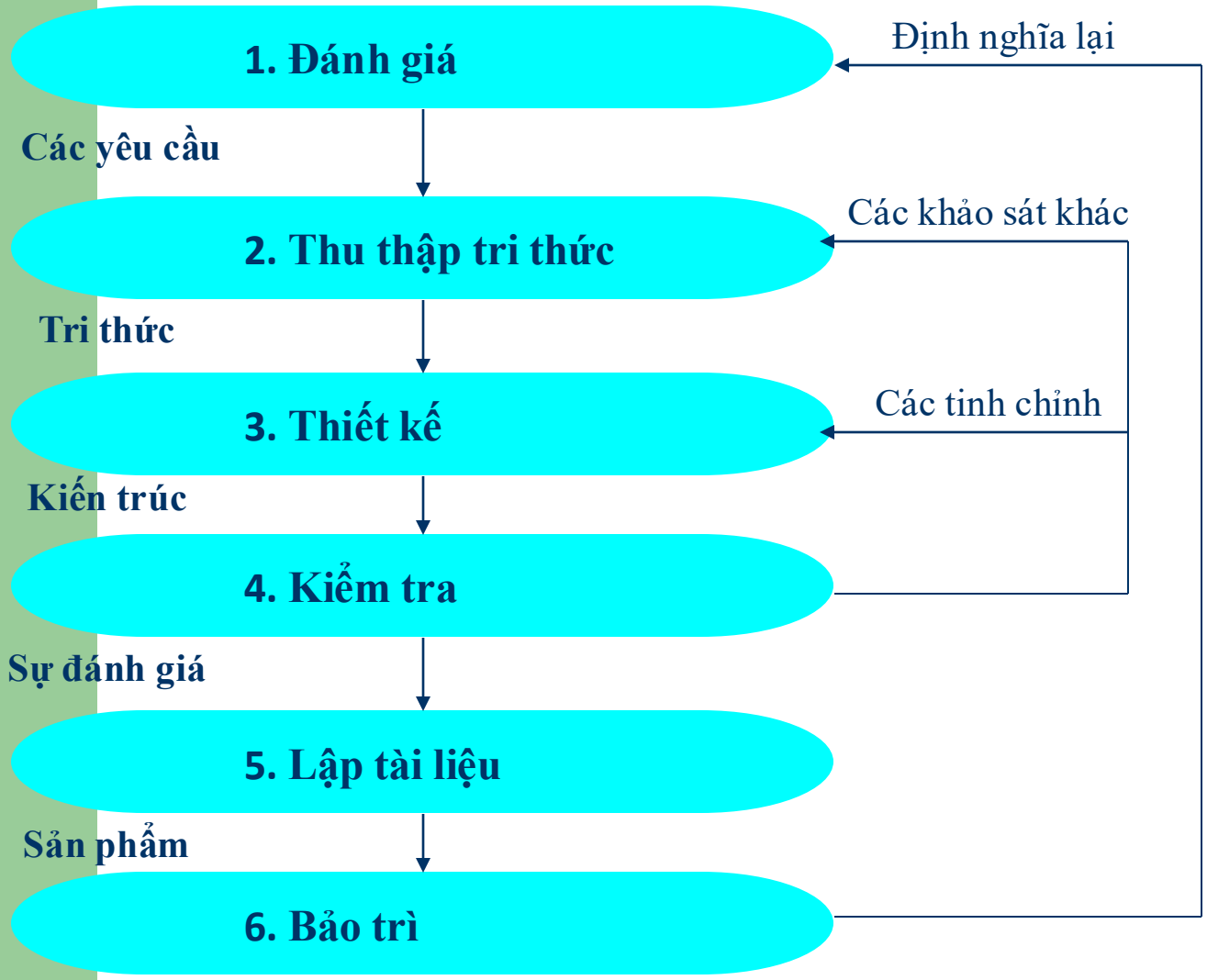
- Có khả năng bị lỗi.

Giống như chuyên gia con người ES có khả năng bị lỗi. Chính vì vậy, cần thiết đưa vào khả năng phục hồi lại lỗi cho ES – ES có khả năng lưu vết quá trình suy luận, nếu nó đưa ra một kết luận mà người dùng kiểm nghiệm với thực tế có sai và báo cho ES, lúc đó nó phải có khả năng ghi nhận và theo đuổi một hướng suy luận khác.

đặc điểm này không xuất hiện trong các chương trình truyền thống, nhưng đừng vội kết luận loại chương trình đó tốt hơn. Mỗi loại có những đặc điểm riêng như bảng so sánh sau:

CT truyền thống	**ES**
Xử lý số	**Xử lý ký hiệu.**
Giải thuật	**Heuristic**
Tích hợp thông tin+ điều khiển	**Tách bạch thông tin+ điều khiển**
Khó thay đổi	**dễ thay đổi.**
Thông tin chính xác	**Thông tin không chắc chắn.**
Giao diện lệnh điều khiển	**Hội thoại + giải thích.**
Kết quả cuối cùng	**đề nghị + giải thích**
Tối ưu	**Có thể chấp nhận.**

Công nghệ tri thức.



- Quá trình gồm các giai đoạn như hình bên.
- Một số định nghĩa:

  ♣ *Công nghệ tri thức:* ***Là quá trình xây dựng ES.***

  ♣ Thu thập tri thức: Là quá trình thu thập, tổ chức và nghiên cứu tri thức.

Các nhân tố trong một dự án ES

- **Các nhân tố chính:**
  - Chuyên gia lĩnh vực.
  - Kỹ sư tri thức
  - Người dùng sản phẩm

- **Các yêu cầu cho mỗi nhân tố:**

Chuyên gia lĩnh vực:

- Có tri thức chuyên gia
- Có kỹ năng giải quyết vấn đề hiệu quả
- Có thể chuyển giao tri thức
- Không chống đối (thân thiện).

Kỹ sư tri thức:

- Có kỹ năng về công nghệ tri thức
- Có kỹ năng giao tiếp tốt.
- Có thể làm cho vấn đề được giải quyết bởi phần mềm.
- Có kỹ năng lập trình hệ chuyên gia.

Người dùng sản phẩm:

- Có thể trợ giúp thiết kế giao diện cho ES.
- Có thể trợ giúp việc thu thập tri thức.
- Có thể trợ giúp trong quá trình phát triển ES.

Các kỹ thuật suy luận

- Suy luận: là quá trình làm việc với tri thức, sự kiện, chiến lược giải toán để dẫn ra kết luận.
- Bạn suy luận như thế nào?

Các hình thức cơ bản:

- ♣♣ Suy luận diễn dịch.
- ♣♣ Suy luận quy nạp.
- ♣♣ Suy luận tương tự.
- ♣♣ Suy luận khả sai.
- ♣♣ Suy luận common-sense.
- ♣Suy luận đơn điệu
- ♣Suy luận không đơn điệu.

Các kỹ thuật cơ bản:

- ♣ Suy luận tiến (forward-chaing)
- ♣ Suy luận lùi (backward-chaining)

Ưu – nhược điểm của mỗi kỹ thuật

Suy luận tiến – forward chaining

- Ưu điểm:
  - ♣ Làm việc tốt với bài toán có bản chất: gôm thông tin và sau đó tìm xem có thể suy ra cái gì từ thông tin đó.
  - ♣ Có thể dẫn ra rất nhiều thông tin chỉ từ một ít sự kiện ban đầu.
  - ♣ Thích hợp cho một số vấn đề như: *hoạch định, giám sát, điều khiển, diễn dịch.*

- Nhược điểm:
  - ♣ Không có cách để nhận thấy tính quan trọng của từng sự kiện. Hỏi nhiều câu hỏi thừa, vì đôi lúc chỉ cần một vài sự kiện là cho ra kết luận.
  - ♣ Có thể hỏi những câu hỏi không liên quan gì nhau – chuỗi câu hỏi không ăn nhập nhau.

VD:

- Bạn có thân nhiệt cao ?
- Bạn đến VN đã lâu rồi ?
- …

Ưu – nhược điểm của mỗi kỹ thuật

Suy luận lùi – backward chaining

- Ưu điểm:
  - ♣ Làm việc tốt với bài toán có bản chất: thành lập giả thiết , sau đó tìm xem có thể chứng minh được không.
  - ♣ Hướng đến một goal nào, nên hỏi những câu hỏi có liên quan nhau.
  - ♣ Chỉ khảo sát CSTT trên nhánh vấn đề đang quan tâm.
  - ♣ Tốt cho các vấn đề: *chuẩn đoán, kê toa, gỡ rối.*

- Nhược điểm:
  - ♣ Luôn hướng theo dòng suy luận định trước thậm chí có thể dừng và rẽ sang một goal khác.

  → Giải quyết: dùng meta-rule để khắc phục.

  Meta-rule: dùng để hướng không gian tri thức được khảo sát sang một vùng khác.

Khảo sát ES: MYCIN

- **Giới thiệu:**
  - ♣ Là ES về lĩnh vực chuẩn đoán bệnh nhiễm trùng nháu.
  - ♣ Được phát triển vào khoảng năm 1970, tại đại học Standford
  - ♣ Là thước đo cho nhiều ES dựa trên luật sau này.
  - ♣ Được viết trên ngôn ngữ INTERLISP, một nhánh của LISP.
  - ♣ Người ta từ MYCIN, tước bỏ phần CSTT của nó để được một hệ thống EMYCIN (empty MYCIN), dùng để phát triển các ES khác.

- **Các đặc điểm chính:**
  - ♣ Sử dụng kỹ thuật suy luận lùi.
  - ♣ Có khả năng phân tách tri thức và điều khiển.
  - ♣ Có tích hợp Meta-rule.
  - ♣ Có thể dùng khi thiếu thông tin hay thông tin không chính xác.
  - ♣ Có khả năng nhớ được các thông tin trong một phiên làm việc với người bệnh cụ thể.
  - ♣ Dể sử dụng – giao tiếp với bác sĩ qua tiếng anh.
  - ♣ Cung cấp các chức năng giải thích: WHY, HOW.

Khảo sát ES: MYCIN

- Một phiên làm việc cụ thể với MYCIN
  ("Expert System". Author: John Durkin)
  --- đã được Việt hóa.

? Một vài thông tin về bệnh nhân:

1. MYCIN: Họ tên ?
   USER: Jack Durkin.
2. MYCIN: Tuổi ?
   USER: 24
3. MYCIN: Giới tính ?
   USER: Nam
4. MYCIN: Chủng tộc?
   USER: CAUCASIAN

? MYCIN cố gắng xác định bản chất lây nhiễm.

5. MYCIN: Có bất kỳ phép cấy vi khuẩn cho Jack Durkin liên qua đến bệnh hiện tại?
   USER: Có
6. MYCIN: Mẫu vật lấy từ đâu?
   USER: CSF
7. MYCIN: Ngày, giờ thực hiện cấy?
   USER: 7- July-7:00

….

MYCIN: Gần đây Jack Durkin có các triệu chứng như: choáng ván, hôn mê không?

USER: Có.

….

Khảo sát ES: XCON

- Giới thiệu:

  ♣ Là ES về lĩnh vực hướng dẫn cấu hình các hệ thống máy tính của DEC trước khi xuất xưởng.

  ♣ Được phát triển vào khoảng năm 1980, tại đại học CMU

  ♣ Được công bố giảm thời gian cấu hình cho mỗi hệ thống xuống còn 2 phút (so với 25 phút bằng tay.). Tiết kiệm vào khoảng 25 triệu $ cho mỗi năm.

  (Theo "Expert System" – John Durkin)

Hệ chuyên gia dựa trên luật

- **Định nghĩa:**

  Là một chương trình máy tính, xử lý các thông tin cụ thể của bài toán được chứa trong bộ nhớ làm việc và tập các luật được chứa trong CSTT, sử dụng động cơ suy luận để suy ra thông tin mới.

  ES dựa trên luật: có nền tảng xây dựng lá hệ luật sinh – chương trước.

  ES dựa trên luật cũng có những đặc trưng cơ bản như đã nêu trong phần trước cho các ES tổng quát, một vài đặc điểm:

  ♣ Có CSTT chứa các luật.

  ♣ Có bộ nhớ làm việc tạm thời.

  ♣ Có động cơ suy luận.

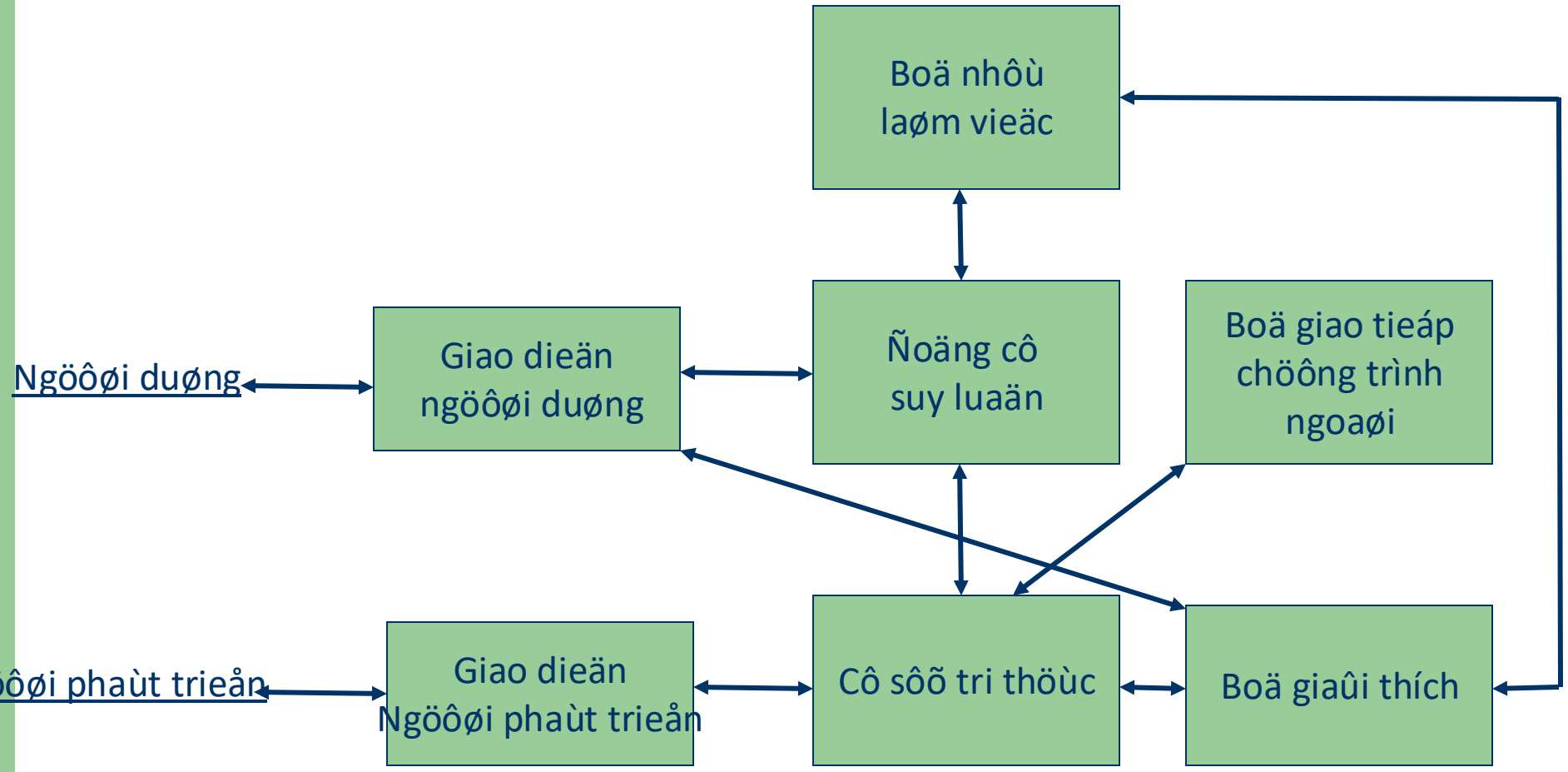
  ♣ Có một giao diện để giao tiếp với người dùng, người phát triển.

  ♣ Có tiện ích giải thích.

  ♣ Có khả năng giao tiếp với chương trình ngoài như: các DBMS, xừ lý bảng tính,…

Hệ chuyên gia dựa trên luật

- Kiến trúc: (như hình sau)
- Nguyên lý hoạt động tương tự hệ luật sinh đã giới thiệu.



Hệ chuyên gia dựa trên luật

- Ưu điểm
  - ♣ Biểu diễn tri thức tự nhiên: IF… THEN.
  - ♣ Phân tách tri thức – điều khiển.
  - ♣ Tri thức là tập các luật có tính độc lập cao -> dễ thay đổi, chỉnh sữa.
  - ♣ Dễ mở rộng.
  - ♣ Tận dụng được tri thức heuristic.
  - ♣ Có thể dùng biến trong luật, tri xuất chương trình ngoài.

- Nhược điểm
  - ♣ Các fact muốn đồng nhất nhau, phải khớp nhau hoàn toàn → Các facts cùng một ý nghĩa phải giống nhau về cú pháp, ngôn ngữ tự nhiên không như vậy.
  - ♣ khó tìm mối qua hệ giữa các luật trong một chuỗi suy luận, vì chúng có thể nằm rải rác trong CSTT.
  - ♣ Có thể hoạt động chậm.
  - ♣ Làm cho nhà phát triển phải hình chung mọi cái ở dạng luật -→ không phải bài toán nào cũng có thể làm được như thế này.

Chương 7: BIỂU DIỄN TRI THỨC

- Biểu diễn tri thức trong AI: vai trò và ứng dụng
- Các kỹ thuật biểu diễn tri thức:
  - Semantic network
  - Lưu đồ phụ thuộc khái niệm
  - Frame
  - Script

Phạm Minh Tuấn

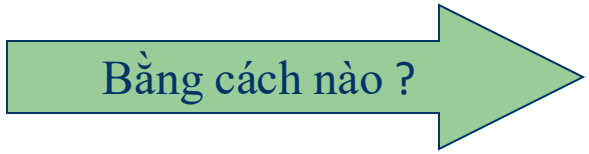
Các lược đồ biểu diễn tri thức.

- Định nghĩa:

  Biểu diễn tri thức là phương pháp để mã hoá tri thức, nhằm thành lập cơ sở tri thức cho các hệ thống dựa trên tri thức (knowledge-based system).



Gồm: đối tượng và các quan hệ giữa chúng trong lĩnh vực.



Bằng cách: dùng các lược đồ biểu diễn (scheme).
→ Chọn dùng lược đồ cho loại tri thức là vấn đề quan trọng.



Gồm: Bảng ánh xạ giữa:
Đối tượng thực → đối tượng tính toán.
Quan hệ thực → quan hệ tính toán.

Các lược đồ biểu diễn tri thức.

- Chú ý:

  Cần phân biệt: Lược đồ biểu diễn (scheme) và môi trường hiện thực (medium), tương tự như việc phân biệt: cấu trúc dữ liệu (CTDL) và ngôn ngữ lập trình. Với một loại CTDL, ví dụ như: Bản ghi (record), chúng ta có hiện thực trong nhiều ngôn ngữ như: Pascal, C,…Tương tự, với một loại lược đồ nào đó chúng ta có thể chọn một trong các NNLT để hiện thực nó.

- Các loại lược đồ biểu diễn:

  ♣ ***Lược đồ logic.***

  Dùng các biểu thức trong logic hình thức ,như phép toán vị từ, để biểu diễn tri thức.

  Các luật suy diễn áp dụng cho loại lược đồ này rất rõ ràng, đã khảo sát trong chương 2 (như: MP, MT,…).

  Ngôn ngữ lập trình hiện thực tốt nhất cho loại lược đồ này là: PROLOG.

  ♣ ***Lược đồ thủ tục:***

  Biểu diễn tri thức như tập các chỉ thị lệnh để giải quyết vấn đề.

Các lược đồ biểu diễn tri thức.

♣ ***Lược đồ thủ tục:***

Ngược lại với các lược đồ dạng khai báo, như logic và mạng, các chỉ thị lệnh trong lược đồ thủ tục chỉ ra bằng cách nào giải quyết vấn đề.

Các luật trong CSTT của ES dựa trên luật là ví dụ về thủ tục giải quyết vấn đề.

Hệ luật sinh là ví dụ điển hình của loại lược đồ này.

♣ ***Lược đồ mạng.***

Biểu diễn tri thức như là đồ thị; các đỉnh như là các đối tượng hoặc khái niệm, các cung như là quan hệ giữa chúng.

Các ví dụ về loại lược đồ này gồm: mạng ngữ nghĩa, phụ thuộc khái niệm, đồ thị khái niệm → được khảo sát sau đây của chương này.

♣ ***Lược đồ cấu trúc:***

Là một mở rộng của lược đồ mạng; bằng cách cho phép các node có thể là một CTDL phức tạp gồm các khe(slot) có tên và trị hay một thủ tục. Chính vì vậy nó tích hợp cả dạng khai báo và thủ tục.

Kịch bản(script) , khung (frame), đối tượng (object) là ví dụ của lược đồ này → khảo sát sau.

Các chú ý về lược đồ.

- Khi xây dựng các lược đồ cần chú ý những vấn đề sau:
  - ♣ Các đối tượng và các quan hệ có thể biểu diễn cho cái gì trong lĩnh vực?

    Ví dụ: để biểu diễn cho ý "Nam cao 1mét 70",chúng ta có thể dùng: ***chieucao(nam,170).*** Vậy thì để diễn tả "An cao hơn Nam" chúng ta làm như thế nào, vì chiều cao của An lúc này không là một trị cụ thể nữa!
  - ♣ Bằng cách nào phân biệt giữa "nội hàm" và "ngoại diện" của một khái niệm.
  - ♣ Bằng cách nào thể hiện được meta-knowledge?
  - ♣ Bằng cách nào thể hiện tính phân cấp của tri thức.

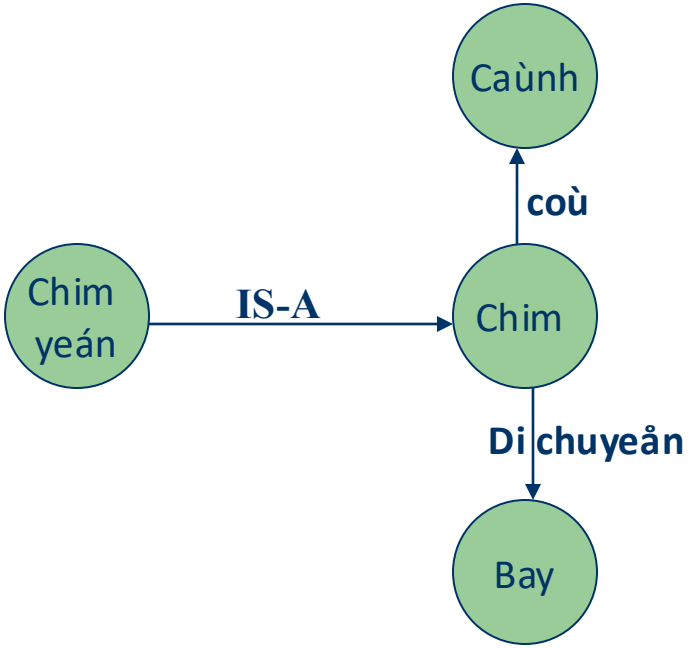
    Lúc biểu diễn tính phân cấp thì các hình thức : kế thừa, ngoại lệ, trị mặc định, ngoại lệ, đa thừa kế phải đặc tả như thế nào
  - ♣ Khi mô tả đối tượng, bằng cách nào có thể tích hợp một tri thức thủ tục vào bản thân mô tả, khi nào thủ tục được thực hiện,..

Mạng ngữ nghĩa

- Định nghĩa:

  Là một lược đồ biểu diễn kiểu mạng, dùng đồ thị để biểu diễn tri thức. Các đỉnh biểu diễn đối tượng; các cung biểu diễn quan hệ giữa chúng.

- Ví dụ:



Xem mạng bên:

- Có hai đỉnh biểu diễn đối tượng,
và hai đỉnh còn lại biểu diễn thuộc tính.
- đỉnh có nhãn: "Chim" nối với hai đỉnh thuộc tính có nhãn: "Cánh", "Bay" nên có thể biểu diễn: "Một con chim thì có cánh và có hình thức di chuyển là bay".
-Đỉnh có nhãn "Chim **yến"** nối với đỉnh "Chim" thông qua cung đặc biệt "IS-**A"** nói lên: "Chim yến là một loài chim".Vì vậy chim yến có thể sỡ hữu các thuộc tính: có cánh, bay như một con chim thông thường.

Mạng ngữ nghĩa

- **Mở rộng mạng ngữ nghĩa:**

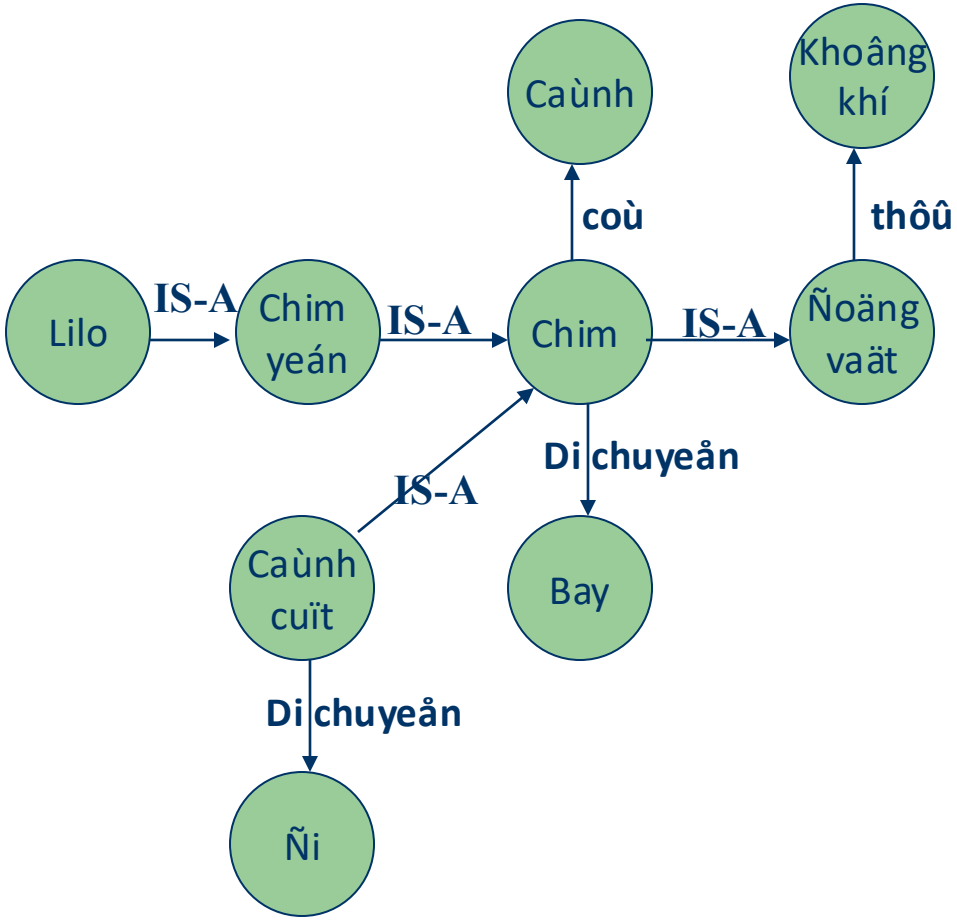
  Để mở rộng mạng thật đơn giản; chúng ta chỉ việc thêm các đỉnh và các cung quan hệ với các đỉnh có sẵn. Các đỉnh được thêm vào mạng hoặc là biểu diễn đối tượng hoặc là biểu diễn thuộc tính như ví dụ trước. Xét ví dụ sau đây minh họa việc mở rộng mạng đã có.

- **Tính thừa kế:**

  Là đặc điểm nổi bật của lược đồ mạng ngữ nghĩa. Mạng ngữ nghĩa định ra cung quan hệ đặc biệt "IS-A" để chỉ ra sự thừa kế. Ví dụ, nhờ tính thừa kế mà từ mạng bên chúng ta có thể suy ra: "Lilo là một động vật có thể bay và hít thở không khí."

- **Tính ngoại lệ:**

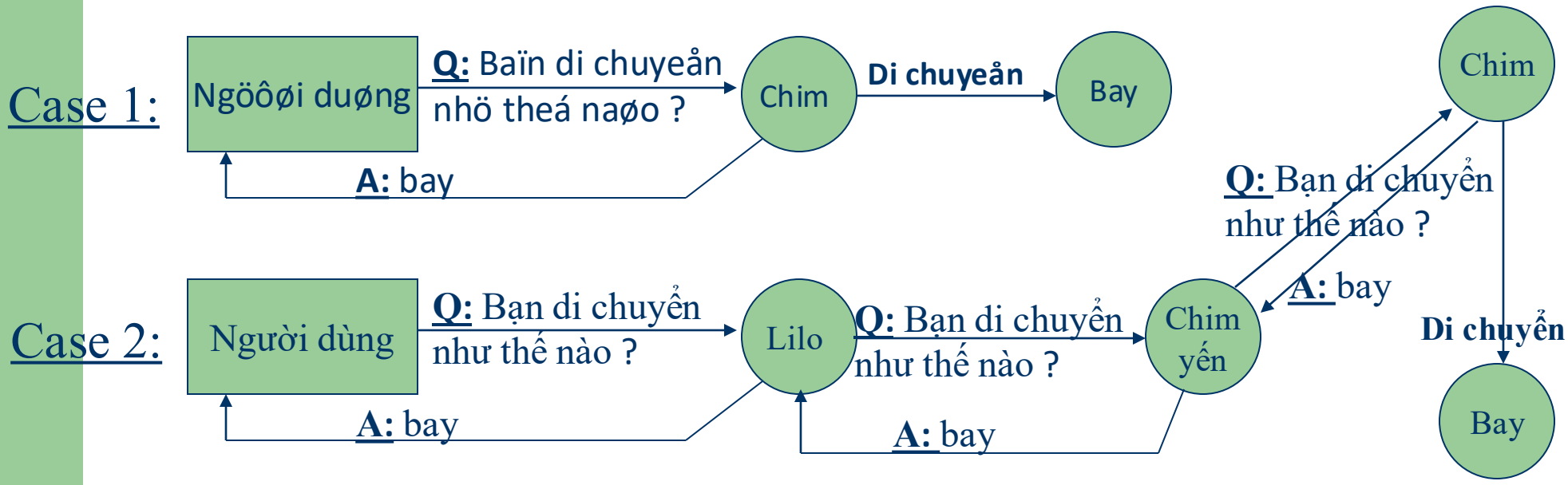
  Định nghĩa một cung quan hệ mới đến một đỉnh có trị khác.



Mạng ngữ nghĩa

- **Phép toán trên mạng ngữ nghĩa:**

  Giả sử chúng ta đã mã hoá mạng ở hình trước vào máy tính. Để dùng mạng, có thể đơn giản là chúng ta câu hỏi với một đỉnh nào đó. Ví dụ, với đỉnh "Chim" chúng ta đặt câu hỏi: "Bạn **di chuyển** như thế nào?". Để trả lời câu hòi chúng ta có thể hiện thực cách trả lời sau cho đỉnh: tìm kiếm cung quan hệ có nhãn "di **chuyển"** bắt đầu từ nó, như case 1,2 ở bên.



Lưu đồ về quan hệ phụ thuộc khái niệm.

- Trong quá trình nghiên cứu về cách hiểu ngôn ngữ tự nhiên, Schank và Rieger đã cố gắng thiết lập một tập các phần tử cơ bản để có thể biểu diễn cấu trúc ngữ nghĩa của các biểu thức ở ngôn ngữ tự nhiên theo một cách đồng nhất.

  Lý thuyết về phụ thuộc khái niệm có đề ra 4 khái niệm cơ bản để từ đó ngữ nghĩa được xây dựng, chúng là:

  - ACT (Action) : các hành động.
    : (các động từ trong câu)
  - PP (Picture Producers) : các đối tượng.
    : (các chủ từ, tân ngữ,..)
  - AA (Action Adder) : bổ nghĩa cho hành động.
    : (trạng từ)
  - PA (Picture Adder) : bổ nghĩa cho đối tượng.
    : (tính từ)

Lưu đồ về quan hệ phụ thuộc khái niệm.

- Tất cả các hành động được cho là có thể được mô tả bằng cách phân rã về một hoặc nhiều hành động như liệt kê sau đây:

1. ATRANS : chuyển đổi một quan hệ – VD: động từ: cho, biếu,…
2. PTRANS : chuyển đổi vị trí vật lý – VD: đi, chạy, di chuyển,..
3. PROPEL : tác động một lực vật lý lên đối tượng – VD: đẩy, chải,…
4. MOVE : di chuyển một phần thân thể bởi đối tượng – VD: đá..
5. GRASP : nắm lấy đối tượng khác. – VD: cầm, nắm, giữ,…
6. INGEST : ăn vào bụng một đối tượng bởi đt khác – VD: ăn, nuốt,..
7. EXPEL : tống ra từ thân thể của một đối tượng – VD: khóc,..
8. MTRANS : chuyển đổi thông tin tinh thần – VD: nói, tiết lộ,..
9. MBUILD : tạo ra một thông tin tinh thần mới – VD: quyết định, …
10. CONC : nghĩ về một ý kiến – VD: suy nghĩ, hình dung,…
11. SPEAK : tạo ra âm thanh – VD: nói, phát biểu,…
12. ATTEND: tập trng giác quan – VD: lắng nghe, nhìn,…

Lưu đồ về quan hệ phụ thuộc khái niệm.

- Quan hệ phụ thuộc khái niệm bao gồm một tập các luật cú pháp cho khái niệm, hình thành nên văn phạm về quan hệ ngữ nghĩa. Các quan hệ này sẽ được dùng vào việc biểu diễn bên trong cho câu trong ngôn ngữ tự nhiên. Danh sách các phụ thuộc khái niệm được liệt kê như bên.



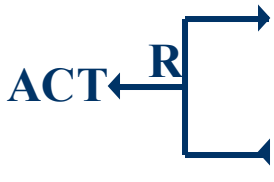
Đối tượng PP
thực hiện hành động ACT



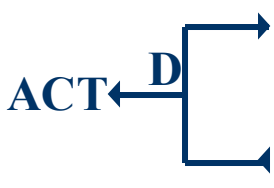
Đối tượng PP có thuộc tính PA



Hành động ACT tác động lên PP



Đối tượng nhận và cho
trong hành động ACT
R: đối tượng nhận (recipient)



Hướng của đối tượng
trong hành động ACT
D: Hướng(Direction)

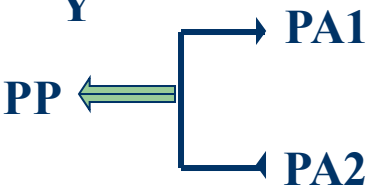


Quan hệ giữa hành động và
thiết bị phục vụ cho hành động.
(xem ví dụ phần sau)

Lưu đồ về quan hệ phụ thuộc khái niệm.



Biểu diễn quan hệ nhân quả.
X: nguyên nhân.
Y: kết quả.

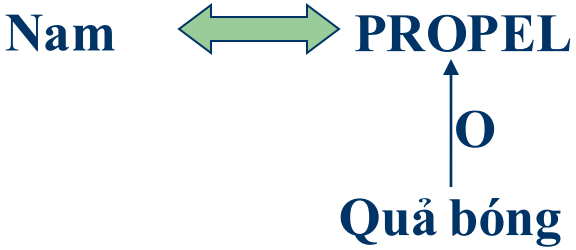


Biểu diễn sự chuyển đổi trạng thái của PP từ PA2 sang PA1



Biểu diễn quan hệ sỡ hữu.
PP2 là PART OF hoặc POSSESSOR OF PP1

♣ Từ các phụ thuộc khái niệm cơ bản nêu trên. Chúng ta có thể kết hợp để có thể biểu diễn các câu trong NNTN, như ví dụ sau:



Ý nghĩa: "Nam đã tác dụng một lực vào quả bóng"

Lưu đồ về quan hệ phụ thuộc khái niệm.

- Các phụ thuộc khái niệm trên cho phép chúng biểu diễn quan hệ giữa: chủ từ với động từ (như phụ thuộc đầu tiên), hay giữa chủ từ và thuộc tính của nó,…. Lược đồ về quan hệ phụ thuộc khái niệm càng đưa ra cách thứa để biểu diễn thì, điều kiện,…, như bên phải.

p : quá khứ – ACT đã xảy ra trong quá khứ

VD:



Ý nghĩa: Nam ***đã*** tác dụng một lực (đẩy) vào cái bàn.

f : tương lai.

t : chuyển tiếp.

ts : bắt đầu chuyển tiếp.

tf : kết thúc chuyển tiếp.

k : đang diễn ra.

? : nghi vấn.

/ : phủ định.

C : điều kiện.

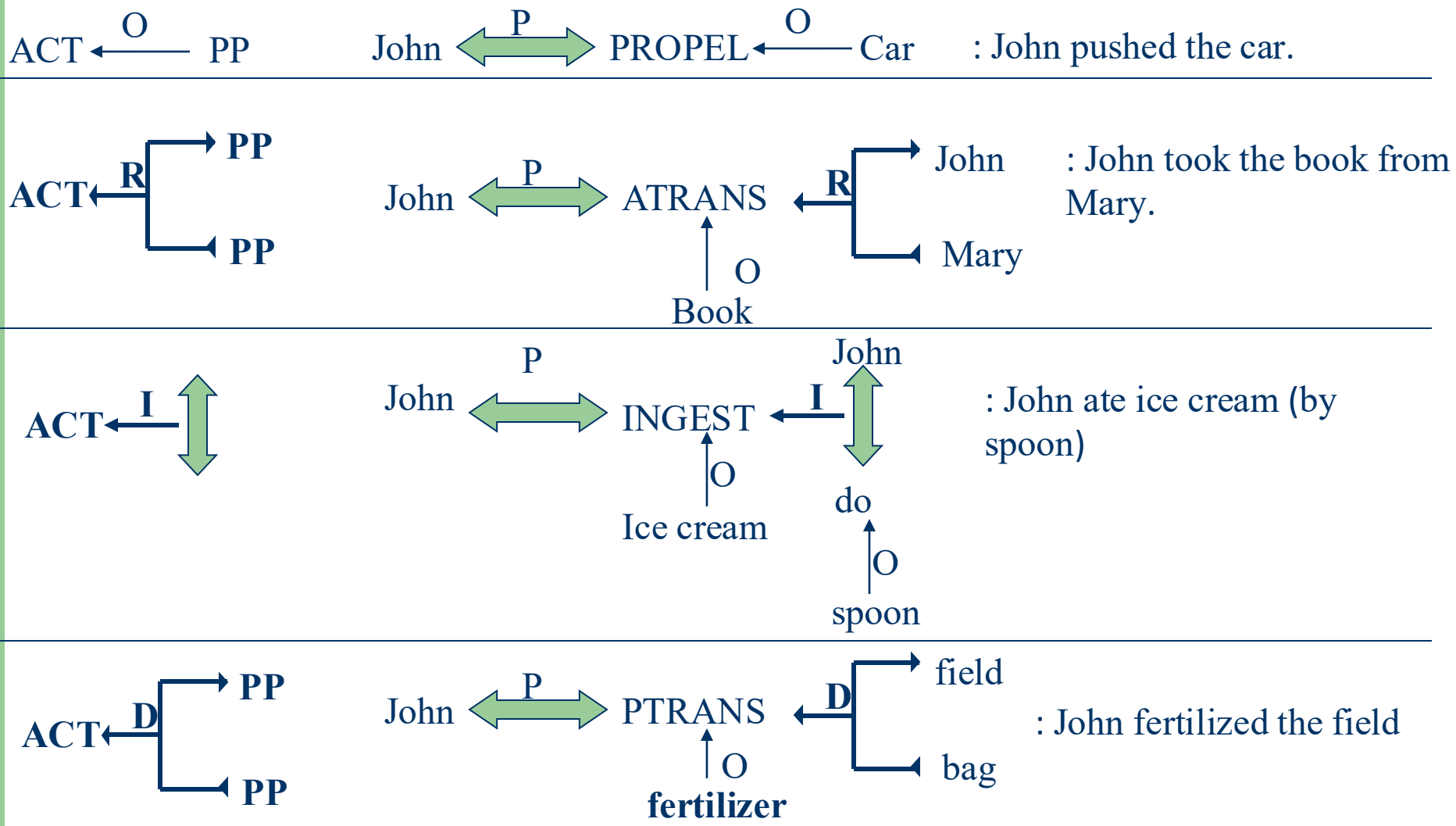
Nil: hiện tại. (không ghi chú gì)

Lưu đồ về quan hệ phụ thuộc khái niệm.

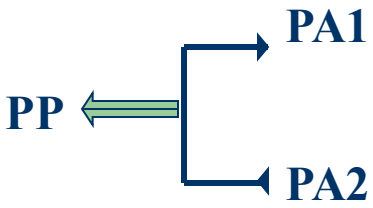
- Một số ví dụ về việc kết hợp các phụ thuộc khái niệm để biểu diễn câu:

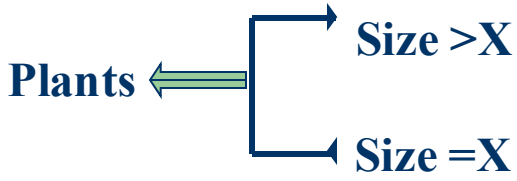
PP ⟺ ACT	**VD:**	Nam ⟺(P) PROPEL ←(O)— Lan	: Nam đã đánh Lan
		Nam ⟺(K) ATTEND ←(O)— Bài giảng	: Nam đang tập trung vào bài giảng.
PP ⟺ PA		Nam ⟺ Height (> average)	: Nam is tall.
PP ⟺ PP		Nam ⟺ doctor	: Nam is a doctor.
PP ←— PA		boy ←— nice	: A nice boy
PP ⟸ PP		dog ⟸(Poss-by) John	: John's dog.

Lưu đồ về quan hệ phụ thuộc khái niệm.

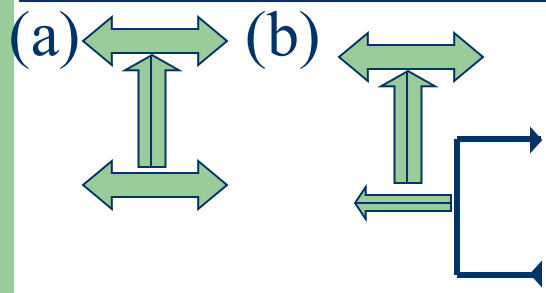


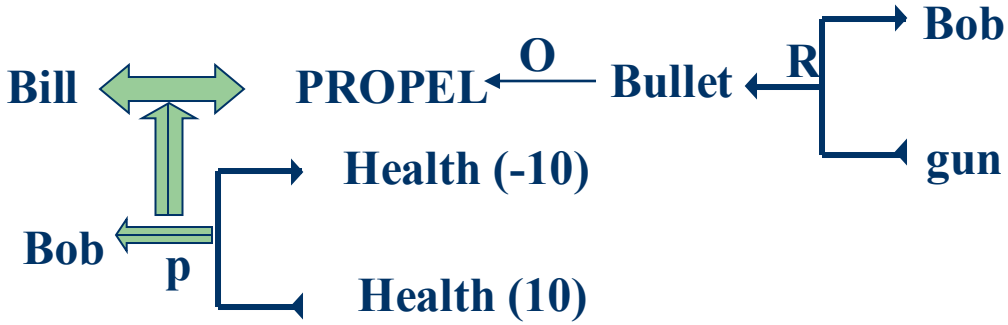
Lưu đồ về quan hệ phụ thuộc khái niệm.



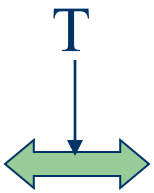


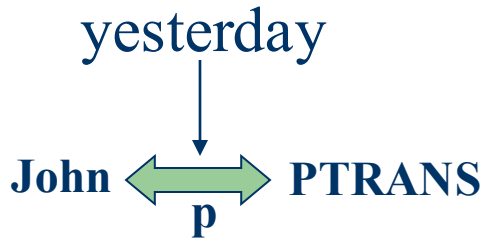
: The plants grew





: Bill shot Bob





: John ran yesterday

Lưu đồ về quan hệ phụ thuộc khái niệm.

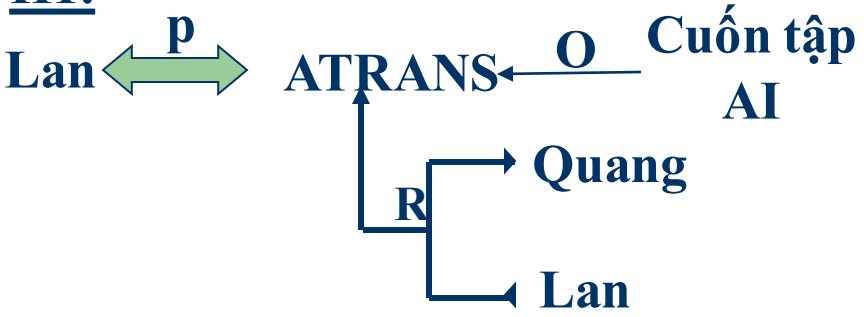
♣ Từ những kết hợp giữa các phụ thuộc khái niệm để biểu diễn các câu đơn giản ở trên, chúng ta có thể cũng có thể tạo ra biểu diễn cho các câu phức tạp hơn như ví dụ sau:

Câu: ***"Nam đã cấm Lan gởi cuốn tập AI cho Quang"***

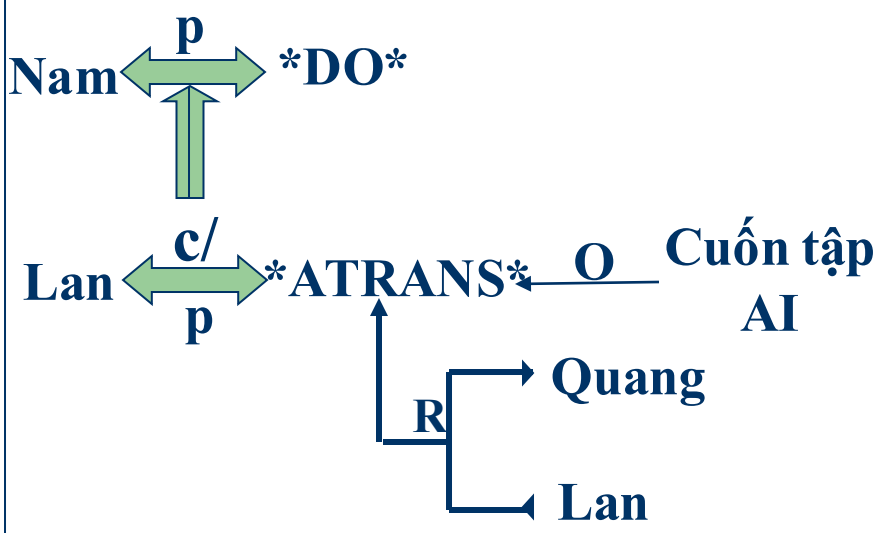
Nếu đặt C là mệnh đề: ***"Lan gởi cuốn tập cho Quang",*** thì câu trên có thể hiểu là: Nam cấm cái mệnh đề vừa nêu xảy ra.

Mà mệnh đề C được biểu diễn như **H1**, nên toàn bộ câu là như **H2**:

H1:



H2:



Lưu đồ về quan hệ phụ thuộc khái niệm.

- Ưu điểm

  ♣ Cung cấp cách thức biểu diễn hình thức cho ngữ nghĩa của ngôn ngữ tự nhiên, ngữ nghĩa được biểu diễn theo dạng có quy tắc → giảm sự nhập nhằng.

  ♣ Chính bản thân dạng biểu diễn chứa đựng ngữ nghĩa → tính **đồng nghĩa** tương ứng là sự **đồng nhất về cú pháp** của lược đồ biểu diễn → chứng minh tính đồng nghĩa ⇔ so trùng hai đồ thị biểu diễn.

- Nhược điểm:

  ♣ Khó khăn trong việc phát triển chương trình để tự động thu giảm biểu diễn của câu bất kỳ về dạng quy tắc chuẩn.

  ♣ Trả giá cho việc phân rã mọi cái về các thành phần cơ bản: ACT, PP, …

  ♣ Các thành phần cơ bản không thích hợp để miêu tả những khái niệm tinh tế của ngôn ngữ tự nhiên, như các từ có nghĩa định tính: cao, đẹp,…

Đồ thị khái niệm

- Định nghĩa:

Đồ thị khái niệm là một đồ thị hữu hạn, liên thông, các đỉnh được chia làm hai loại: đỉnh khái niệm và đỉnh quan hệ.

Đỉnh khái niệm: dùng để biểu diễn các khái niệm cụ thể (cái, điện thoại, …) hay trừu tượng (tình yêu, đẹp, văn hoá,…). Đỉnh khái niệm được biểu diễn bởi hình chữ nhật có gán nhãn là khái niệm.

Đỉnh quan hệ: dùng để chỉ ra quan hệ giữa các khái niệm có nối đến nó.

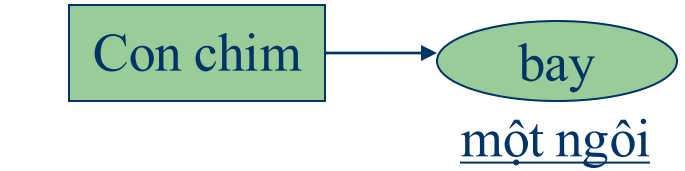
Trong đồ thị khái niệm: chỉ có khác loại mới nối được với nhau. Chính vì dùng đỉnh quan hệ nên các cung không cần phải được gán nhãn nữa.

♣ Mỗi đồ thị khái niệm biểu diễn một mệnh đề đơn.

♣ Cơ sỡ tri thức: chứa nhiều đồ thị khái niệm.

Đồ thị khái niệm

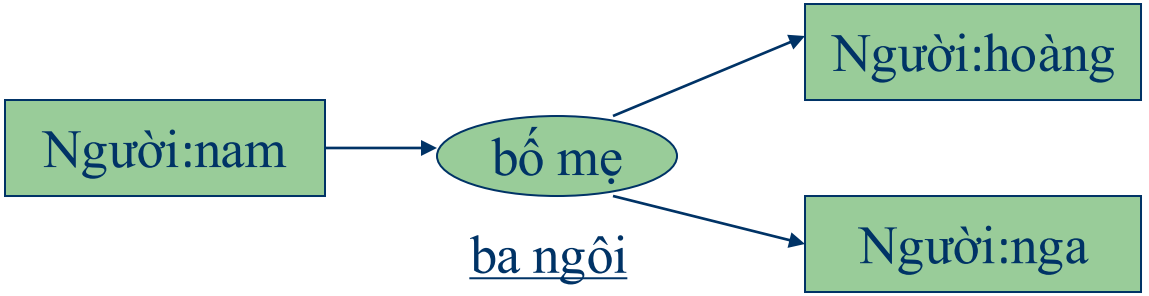
- Một số ví dụ:



Biểu diễn: "Con chim biết bay"



Biểu diễn: "con chó có màu nâu".

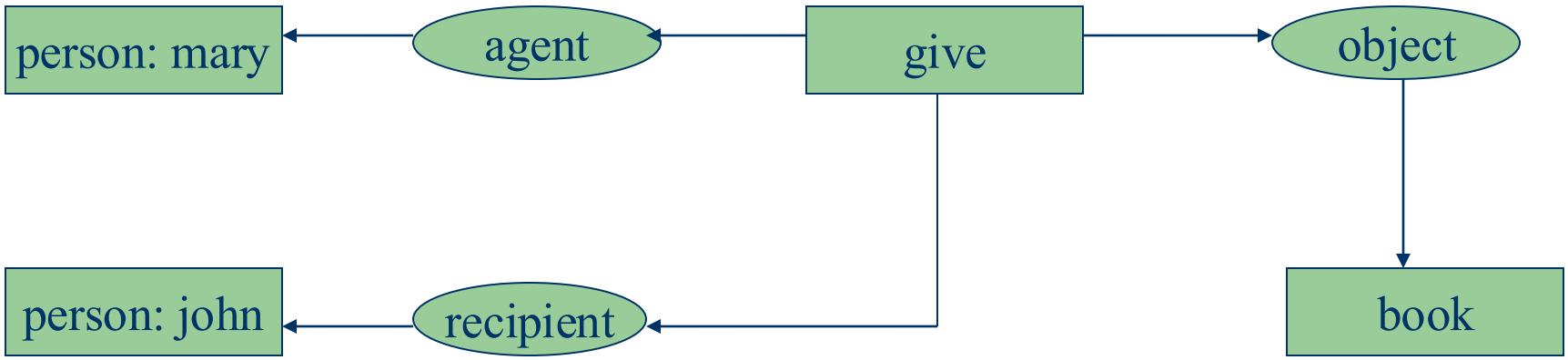


Biểu diễn: "Nam có bố mẹ là ông Hoàng và bà Nga".

** Một đỉnh quan hệ có thể là một hay nhiều ngôi.*

Đồ thị khái niệm

- Một số ví dụ:



Represent: "Mary give John the book"

Trong ví dụ trên, chú ý: động từ "give" có chủ từ thông qua đỉnh quan hệ "agent", tân ngữ trực tiếp thông qua đỉnh quan hệ "object", tân ngữ gián tiếp cũng là người nhận thông quan đỉnh quan hệ "recipient", hướng mũi tên cho các loại động từ tương tự có dạng như đồ thị trên.

Đồ thị khái niệm

- Loại, cá thể, tên:

Trong đồ thị khái niệm, mỗi đỉnh quan hệ biểu diễn cho một cá thể đơn lẽ thuộc một loại nào đó. Để nói lên quan hệ giữa "loại-cá thể", nên mỗi đỉnh khái niệm được quy định cách gán nhãn là:

"loại: tên_cá_thể"

tên_cá_thể có thể là:

1. Một tên nào đó, như:

sinhviên: nam → một sinh viên có tên là Nam.

2. Một khoá để phân biệt, được viết theo cú pháp **#khoá,** như

sinhviên: #59701234 → một sinh viên có khoá là: 59701234.

3. Có thể dùng dấu sao (*) để chỉ ra một cá thể chưa xác định, như:

sinhviên: * , có tác dụng như sinhviên → chỉ ra một sinh viên bất kỳ

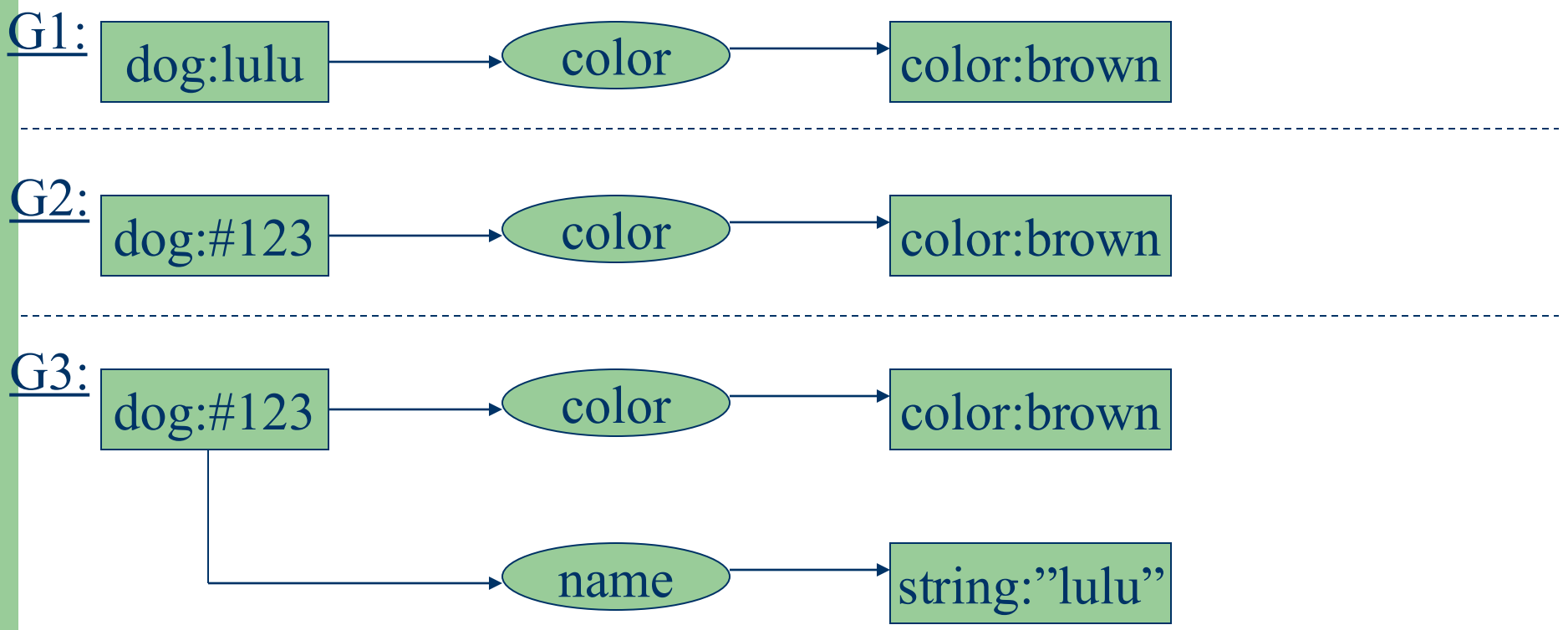
sinhviên:*X → sinh viên bất kỳ, tên sinh viên đã được lấy qua biến X.

sinhviên:ng* → sinh viên có tên bắt đầu bởi "ng"

Trường hợp 1 và 2, khái niệm được gọi là khái niệm cá thể, trường hợp 3 ta có khái niệm tổng quát.

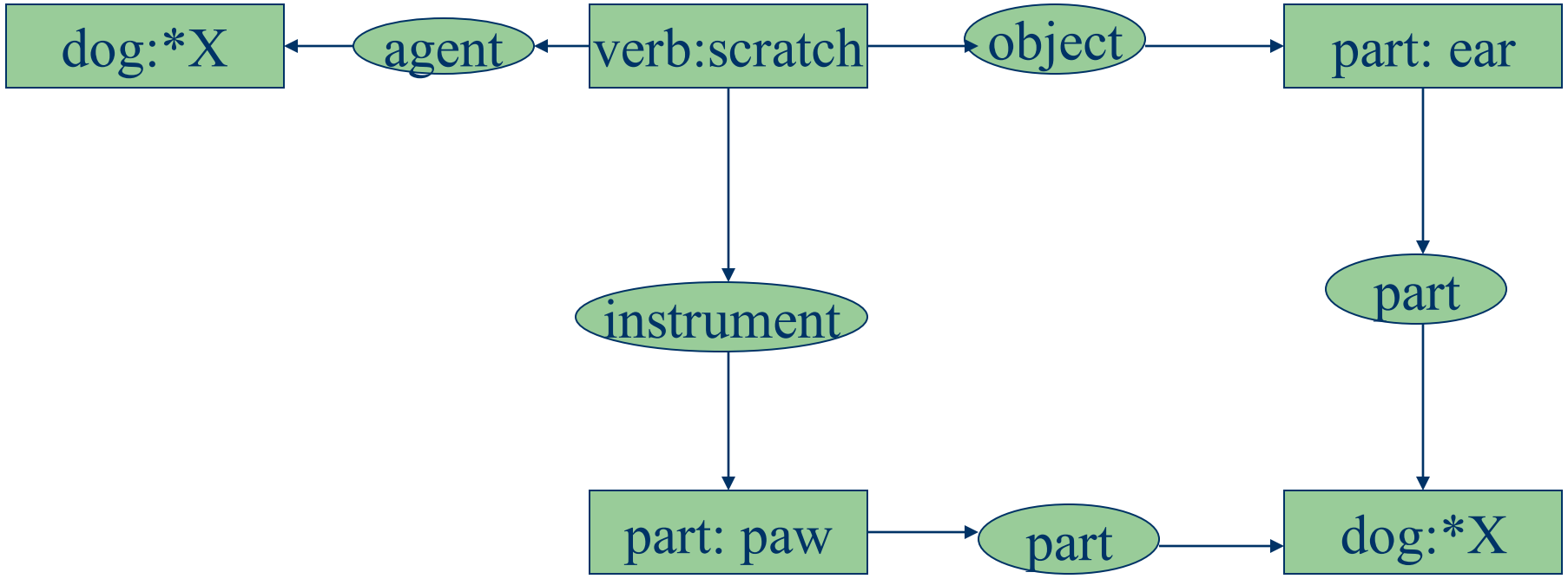
Đồ thị khái niệm

- Nếu dùng cách đặt tên như nói trên có thể nhín thấy 3 đồ thị sau có tác dụng biểu diễn như nhau nếu con có luu có khoá là #123.



Đồ thị khái niệm

- Biến có thể được dùng khi cần chỉ ra nhiều đỉnh khái niệm đồng nhất nhau trong một đồ thị như trường hợp sau.



Represent: "The dog scratches its ear with its paw"

Đồ thị khái niệm

- Phân cấp loại (type)

  Nếu có s và t là hai loại (type) thì:

  s ≤ t : → s: **subtype** của t

  → t : **supertype** của s

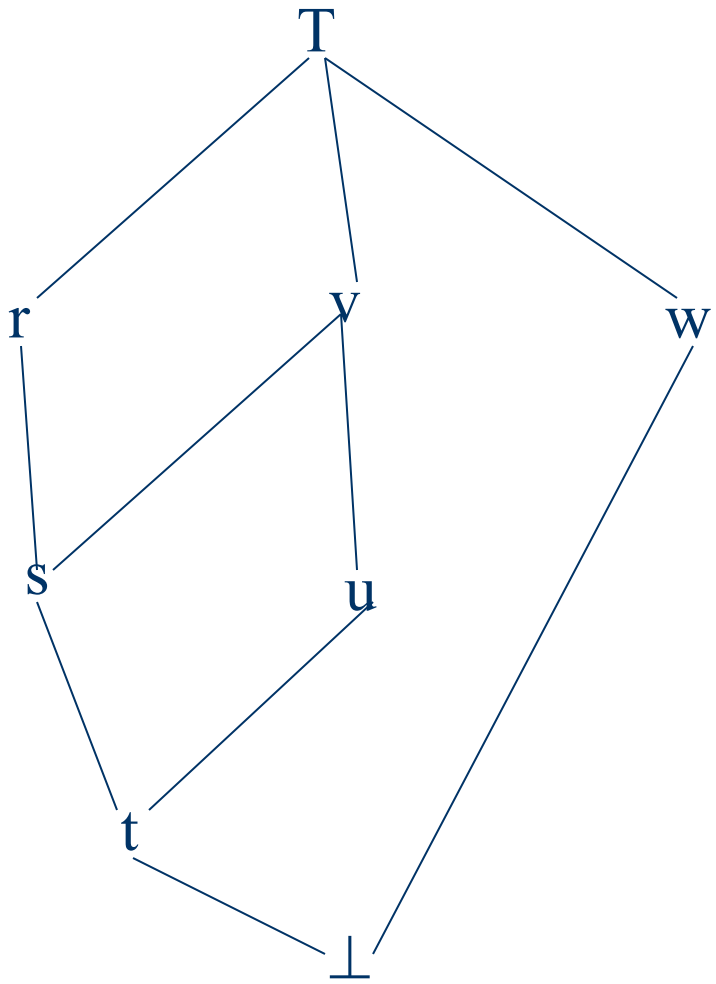
Ví dụ:

- sinhviên là subtype của người.
- người là super type của sinhviên.

→ nên viết: ***sinhviên ≤ người***

Trong sơ đồ phân cấp bên,

- s: được gọi là **common-subtype** của r và v.
- v : được gọi là **common-supertype** của s và u.
- T : supertype của mọi type
- ⊥ : subtype của mọi type

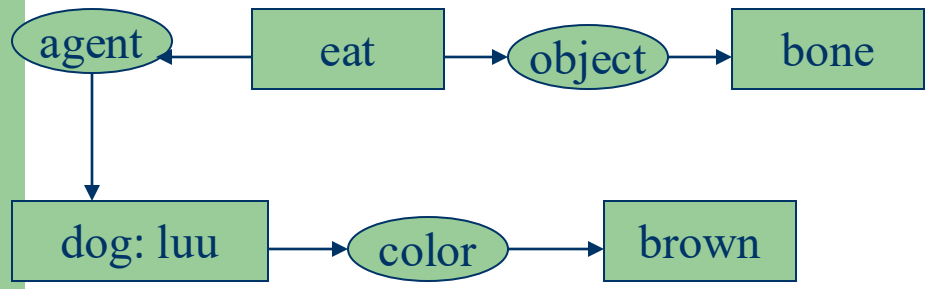


Đồ thị khái niệm

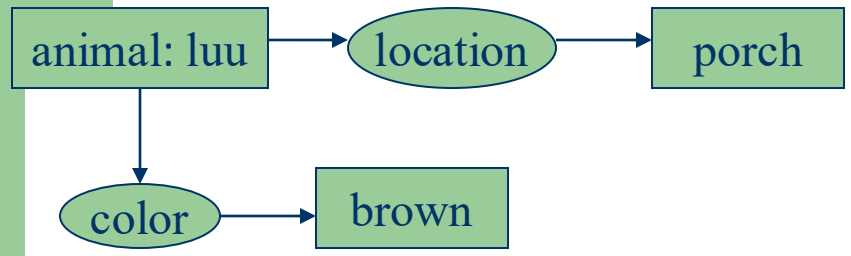
- Các phép toán trên đồ thị khái niệm.

  Xét hai đồ thị sau:

G1:



G2:



♣**Phép copy (nhân bản):** nhân bản một đồ thị.

♣**Phép Restriction (giới hạn):** tạo ra đồ thị mới bằng cách: từ một đồ thị đã có, thay thế một đỉnh khái niệm bởi một đỉnh khác cụ thể hơn, như hai trường hợp:

→ Một biến *, được thay thế bởi một khoá, hay một tên của cá thể.

VD: ***dog:*** → ***dog:#123*** hay ***dog:luu***

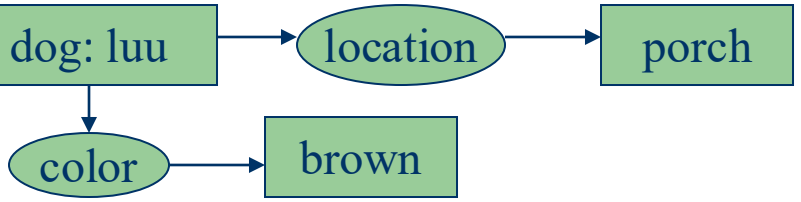
→ Một type được thay thế bởi subtype của nó.

VD: ***người: nam*** → ***sinhviên:nam***

Đồ thị khái niệm

Aùp dụng phép restriction trên đồ thị G2, có thể dẫn ra G3 như sau:

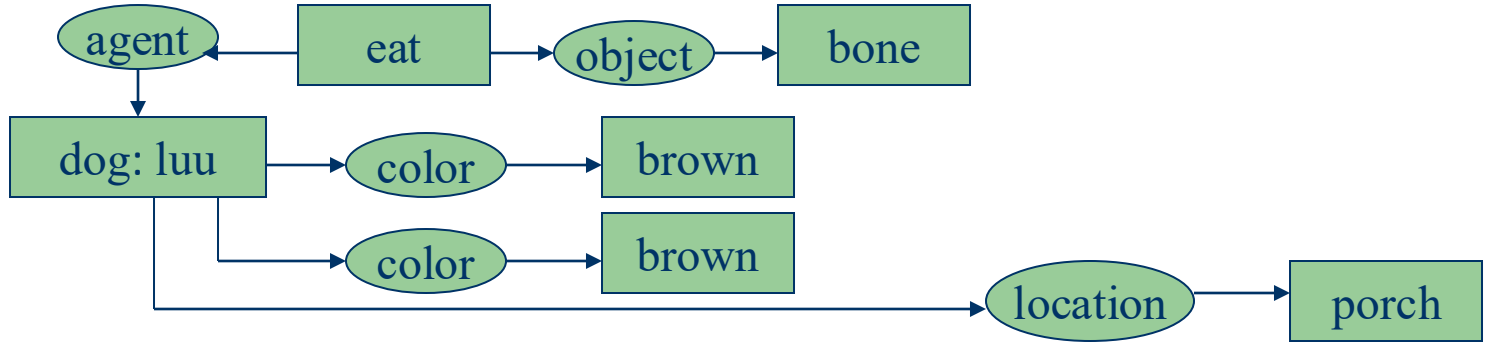
G3:



♣ **Phép Join (nối):** Nối hai đồ thị để được một đồ thị khác.

Nếu có đỉnh khái niệm C xuất hiện trên cả hai đồ thị X và Y, thì chúng ta có thể nối hai đồ thị trên đỉnh chung C nói trên, như từ G1 và G3 có thể tạo ra G4 như sau: (nối trên đỉnh chung là: *dog:lulu*)

G4:

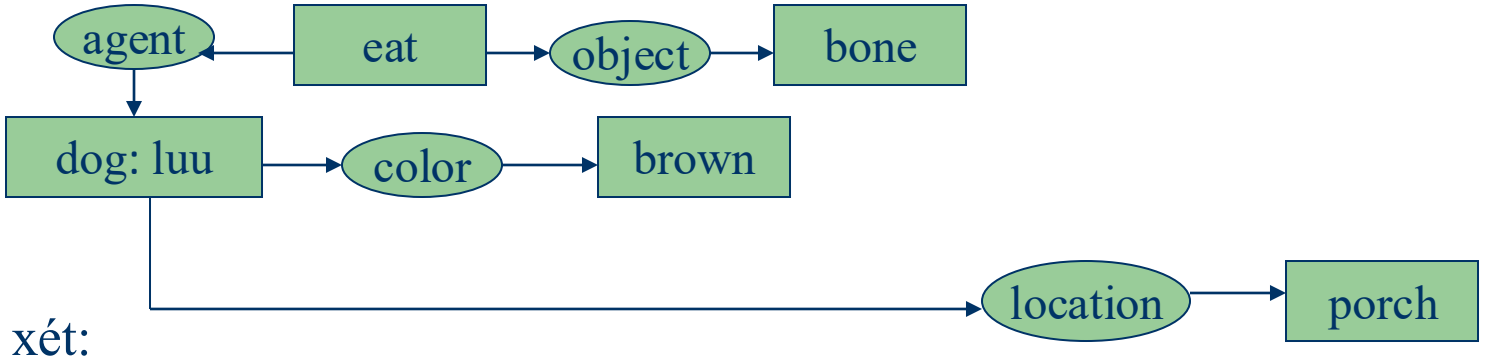


Đồ thị khái niệm

♣ **Phép simplify: (rút gọn)**

Nếu trên một đồ thị có hai đồ thị con giống nhau hoàn toàn thì chúng ta có thể bỏ đi một để tạo ra một đồ thị mới có khả năng biểu diễn không thay đổi. Từ G4 có thể sinh ra G5 cùng khả năng biểu diễn.

G5:



Nhận xét:

Phép Restriction và phép Join cho phép chúng ta thực hiện tính thừa kế trên đồ thị khái niệm. Khi thay một biến * bởi một cá thể cụ thể, lúc đó chúng ta cho phép cá thể thừa kế các tính chất từ loại(type) của nó, cũng tương tự khi ta thay thế một type bởi subtype của nó.

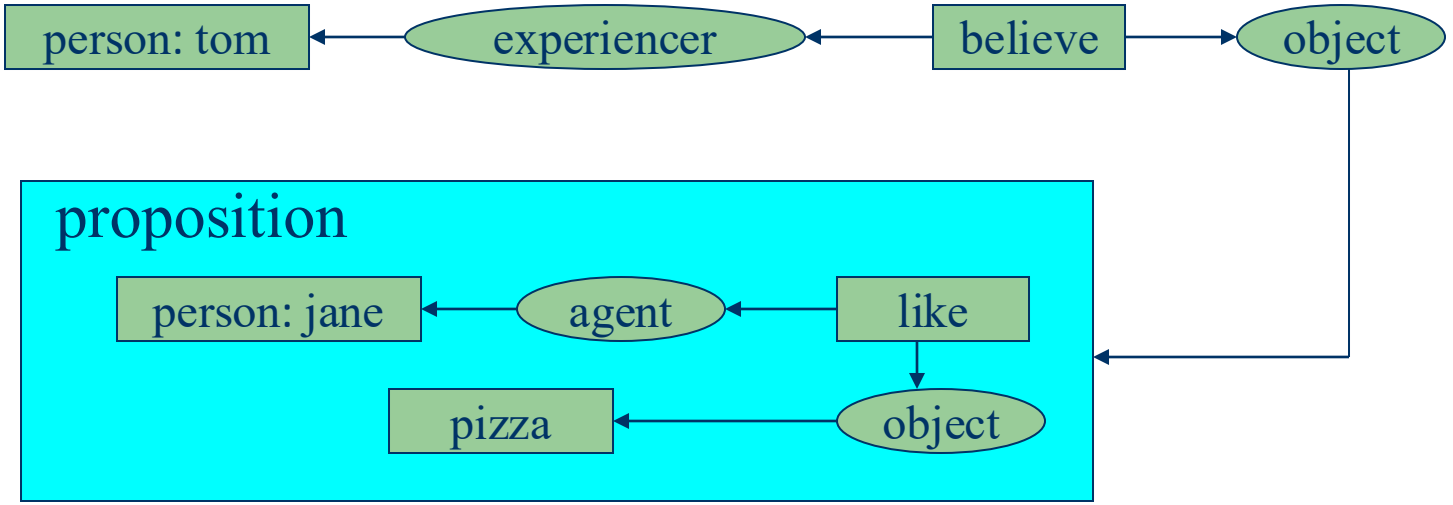
Đồ thị khái niệm

- Đỉnh mệnh đề:

  Để thuận tiện biểu diễn cho các câu gồm nhiều mệnh đề, đồ thị khái niệm đã được mở rộng để có thể chứa cả một mệnh đề trong một đỉnh khái niệm, lúc đó chúng ta gọi là đỉnh mệnh đề.

  Vậy đỉnh mệnh đề là một đỉnh khái niệm có chứa một đồ thị khái niệm khác. Xét đồ thị khái niệm mở rộng biểu diễn cho câu:

  "Tom believes that Jane likes pizza".

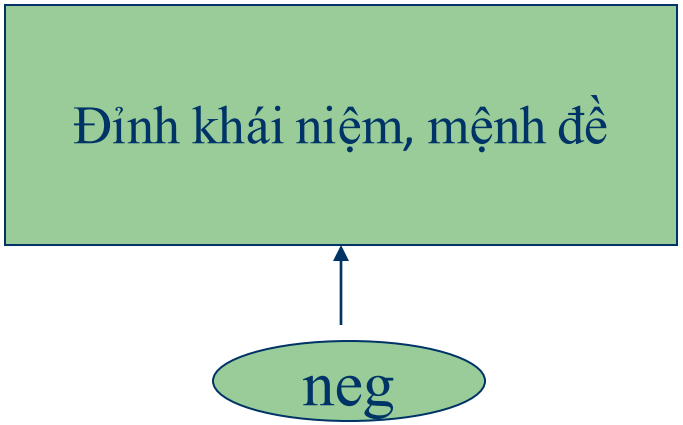


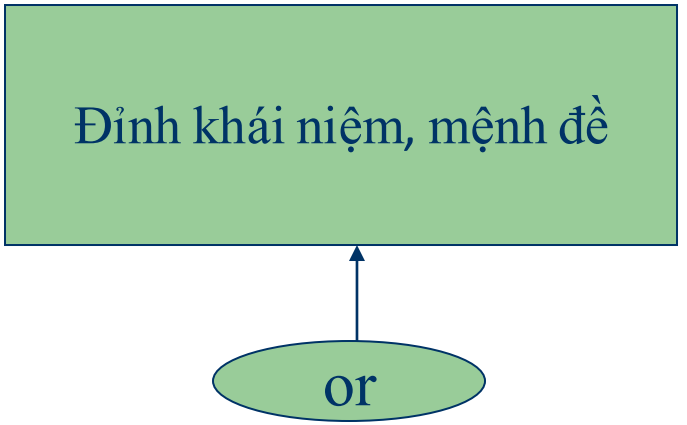
Đồ thị khái niệm

- Đồ thị khái niệm và logic.

  - Phép hội (and) của nhiều khái niệm, mệnh đề chúng ta có thể thực hiện dễ dàng cách cách nối nhiều đồ thị bởi phép toán join.

  - Phép phủ định(not) và phép tuyển(or) giữa các khái niệm hay mệnh đề cũng có thể được thể hiện bằng cách đưa vào đỉnh quan hệ có tên: neg(phủ định), or(tuyển) như dạng sau.

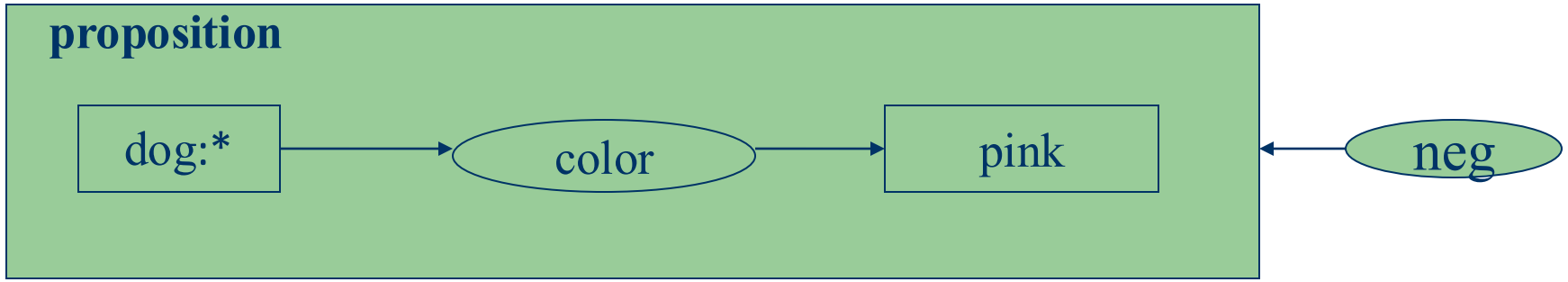




Đồ thị khái niệm

Ví dụ:

Câu: “There are no pink dogs”, được biểu diễn:



Trong đồ thị khái niệm, các khái niệm tổng quát (đỉnh dùng biến * - như ***dog:****, hay chỉ có tên loại - như ***dog***) được xem như có lượng từ tồn tại (∃). Do vậy, mệnh đề trong ví dụ trên có biểu diễn vị từ là:

∃X∃Y(dog(X) ^ color(X,Y) ^ pink(Y)).

Và toàn bộ đồ thị (bao gồm đỉnh quan hệ :neg), có biểu diễn vị từ:

¬ ∃X∃Y(dog(X) ^ color(X,Y) ^ pink(Y)).

= **∀X∀Y(¬ (dog(X) ^ color(X,Y) ^ pink(Y))).**

Đồ thị khái niệm

- Giải thuật để chuyển một đồ thị khái niệm sang biểu diễn vị từ:

1. Gán một biến riêng biệt (X1, X2,…) cho mỗi khái niệm tổng quát.
2. Gán một hằng cho mỗi khái niệm cá thể trong đồ thị. Hằng này có thể là tên cá thể hay khoá của nó.
3. Biểu diễn một đỉnh khái niệm bởi một vị từ một ngôi; có tên là tên loại (type), đối số là biến hay hằng vừa gán trên.
4. Biểu diễn mỗi đỉnh quan hệ bởi một vị từ n ngôi; có tên là tên của đỉnh quan hệ, các thông số là biến hay hằng được gán cho các đỉnh khái niệm nối đến nó.
5. Hội của tất cả các câu trong bước 3 và 4. Tất cả các biến trong biểu thức thu được đều đính kèm lượng từ tồn tại.

Ví dụ: có đồ thị như sau



Được chuyển sang là:

∃X1(dog(luu) ^ color(X1) ^ brown(X1))

Lược đồ có cấu trúc - Frame

- Frame – khung.

  Là một cấu trúc dữ liệu cho phép biểu diễn tri thức ở dạng khái niệm hay đối tượng.

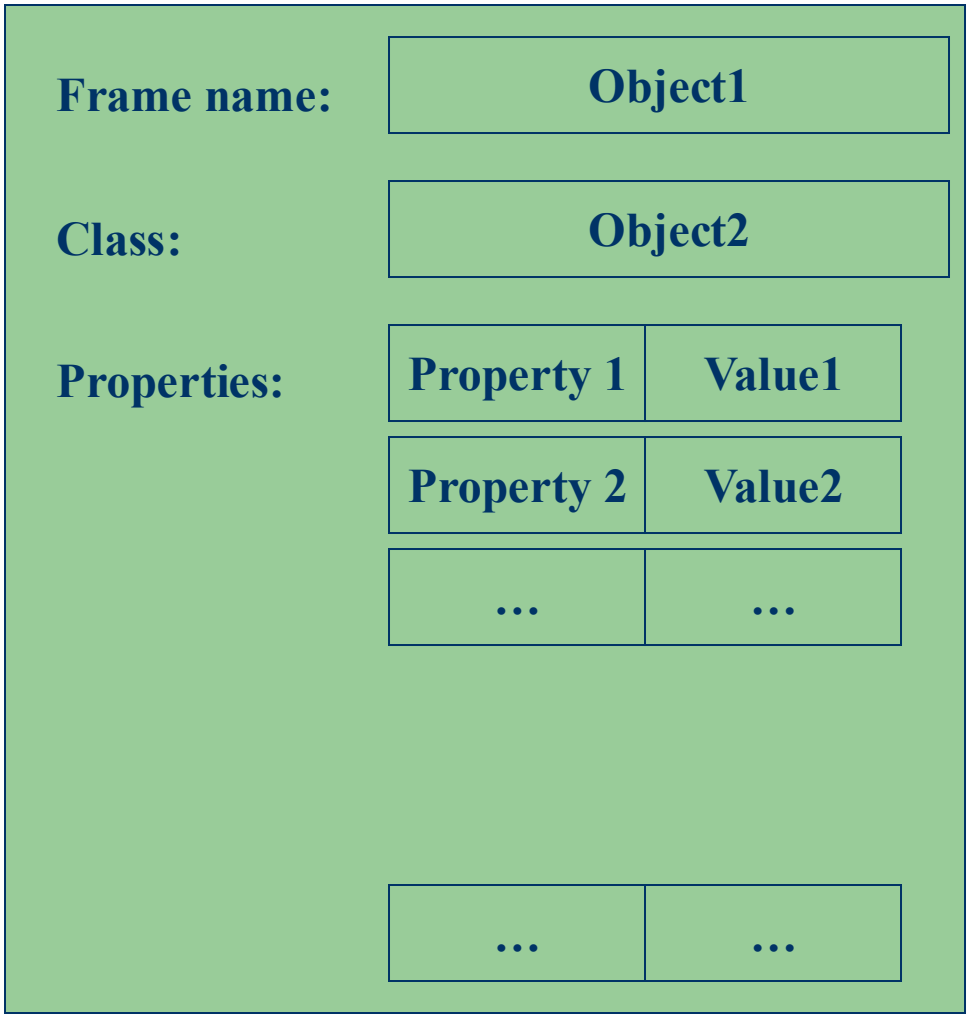
  Một khung có cấu trúc như hình vẽ bên.

- Cấu trúc của frame:

  Đặc tả cho một frame gồm các thành phần cơ bản sau:

  1. Frame name: tên của frame.

  - Nếu frame biểu diễn cho một cá thể nào đó, thì đây là tên của cá thể. Ví dụ: an, nam, lulu,..



Lược đồ có cấu trúc – Frame

- Cấu trúc của frame (tt):

  - Nếu Frame biểu diễn cho một lớp, thì đây là tên lớp. Ví dụ: chim, động vật, …

  2. Class: Tên loại.

  - Nếu thành phần này xuất hiện, nó cho biết rằng frame mà chúng ta đang biểu diễn có loại là giá trị trường class. → ***Cho phép thành lập quan hệ thừa kế IS-A.***

  Như ví dụ trên, chúng ta có:

  Object1 IS-A Object2

3. Các thuộc tính (property): Khi biểu diễn một frame chúng ta có thể thiết lập một hay nhiều thuộc tính cho nó, như ví dụ sau:

Frame name: **chim**

Properties:

maøu	Chöa bieát
aên	Coân truøng
Soá caùnh	2
bay	true
ñoùi	Chöa bieât
Hoaït ñoäng	Chöa bieât

Lược đồ có cấu trúc – Frame

- Cấu trúc của frame (tt):

  - Khi chúng ta đặt tả thuộc tính cho một lớp; nếu chúng ta biết được giá trị chung cho tất cả các đối tượng thuộc lớp mà chúng ta đang biểu diễn thì điền vào trị cho thuộc tính đó, giá trị đó chúng ta gọi là ***giá trị mặc nhiên, như: ăn, số cánh ở trên*** ; nếu chúng ta chưa biết trị cụ thể (nhưng biết là có thuộc tính đó) thì chúng ta có thể bỏ trống ***(chưa biết) – như màu, hoạt động,..***:.

Các thuộc tính của frame nằm ở hai dạng cơ bản:

→ *Dạng tĩnh(static):* giá trị của nó không thay đổi trong quá trình hệ thống tri thức hoạt động.

→ *Dạng động(dynamic):* giá trị có thể chuyển đổi.

Khi phải tìm kiếm một frame, chúng ta có thể dựa vào ***frame name*** , cũng có thể dựa vào các thuộc tính được đặt tả cho frame.

Lược đồ có cấu trúc – Frame

- Cấu trúc của frame (tt):

4. <u>Các thủ tục:</u> Lược đồ frame càng cho phép tích hợp cách thức đặt như các thuộc tính như trên và các thủ tục vào một frame. Về hình thức, một thủ tục sẽ chiếm một khe tương tự như khe thuộc tính nói trên.

Thủ tục được dùng để: biểu diễn một hành động nào đó của đối tượng, điều khiển giá trị của thuộc tính như: kiểm tra ràng buộc về trị, kiểu, của thuộc tính mỗi khi cần trích, hay thay đổi nó.

Hai thủ tục phổ biến được đính kèm với một thuộc tính là:

IF_NEEDED và

IF_CHANGED.

→ *<u>IF_NEEDED:</u>*

Thủ tục này được thực thi mỗi khi chúng ta cần đến giá trị của thuộc tính (giống thủ tục GET trong VB).

Ví dụ: thủ tục sau (dạng if_needed) cho thuộc tính ***bay*** của frame ***chim*** nói trên.

```
If      self:số_cánh < 2
Then self:bay = false
If      self:số_cánh = 2
Then self:bay = true
```

Lược đồ có cấu trúc – Frame

- Cấu trúc của frame (tt):

→ *IF_CHANGED:*

Thủ tục này được thực thi mỗi khi giá trị của thuộc tính mà if_changed này được gắn vào thay đổi. (giống như SET, LET trong VB)

Ví dụ: gắn thủ tục sau cho thuộc tính ***đói*** của lớp ***chim*** nói trên.

If Seft:đói = true

Then Seft:hànhđộng = eating # seft:ăn

4. Các thông tin khác:

Một số khe khác của frame có thể chứa frame khác, link đến frame, mạng ngữ nghĩa, rules, hay các loại thông tin khác.

Chú ý: các ví dụ trên mô phỏng theo ngôn ngữ *Kappa PC*, trong đó, "Expert ***System - DurKin"***:

- Seft: từ khoá chỉ chính bản thân frame đang mô tả (như Me của VB, this của VC)

- # : dấu nối chuỗi(như & của VB, + của VC)

- Lược đồ frame cũng giống như các hệ thống hướng đối tượng. Chúng ta:

→ Có thể đặt tả frame lớp hay cá thể.

→ Có thể đặt tả tính thừa kế.

→ Mỗi khi tạo ra frame cá thể, có thể copy các thuộc tính, thủ tục của frame lớp; đồng thời có thể mở rộng thêm, hay định nghĩa lại một số thuộc tính, thủ tục.

Lược đồ có cấu trúc – Script

- Script – Kịch bản:

  Là một lược đồ biểu diễn có cấu trúc, dùng để biểu diễn một chuỗi các sự kiện trong một ngữ cảnh cụ thể. Nó như một phương tiện để tổ chức các phụ thuộc khái niệm – (đã giới thiệu trước) để mô tả một tình huống cụ thể.

  Script được dùng trong các hệ thống hiểu NNTN, tổ chức tri thức trong thành phần các tình huống mà hệ thống phải tìm hiểu.

- Cấu trúc của Script:

1. Entry conditions:

Các điều kiện phải true để script được gọi. Ví dụ: một cá nhân bị bệnh thì script nhập viện được gọi.

2. Results:

Kết quả thu được từ script khi nó hoàn thành.

3. Props:

Các đồ vật tham gia vào script, như: xe cứu thương, cán, bình oxy,…

4. Roles:

Các cá nhân tham gia vào script, như: bệnh nhân, bác sĩ, y tá, người nhà,…

5. Scenes:

Các cảnh chính trong script, như: di chuyển, cấp cứu, hồi sức,..

→ Một ví dụ về kịch bản đi nhà hàng như ví dụ sau:

Lược đồ có cấu trúc – Script

Script: RESTAURENT
Track: Coffe Shop
Entry conditions:
S is hungry
S has money
Results:
S has less money
O has more money
S is not hungry
S is pleased (optional)
Props: Tables
Menu
Food (F)
Check
Money
Roles: Custumer (S)
Waiter(W)
Cook(C)
Cashier(M)
Owner(O)

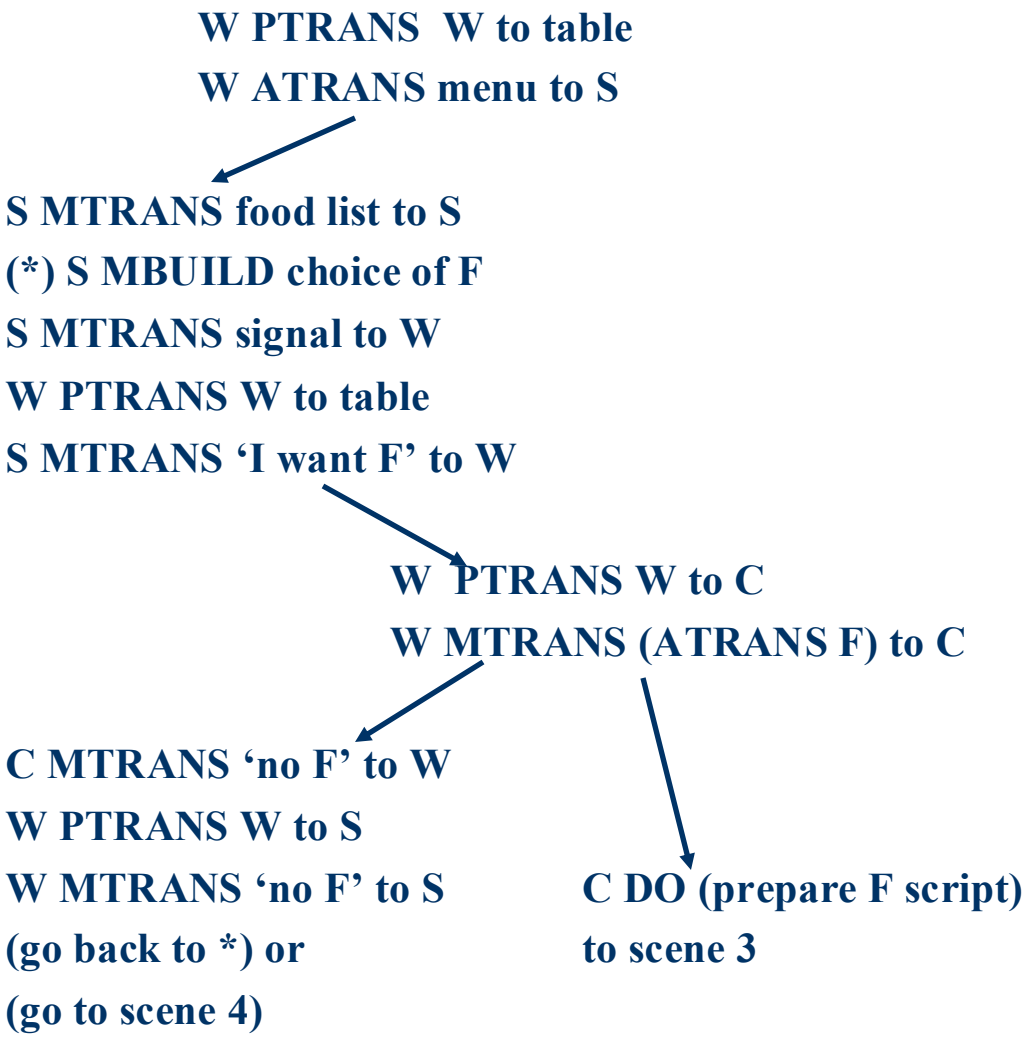
Scene 1: (Entering)

S PTRANS S into restaurent.
S ATTEND eyes to tables
S MBUILD where to sit
S PTRANS S to table
S MOVE S to sitting position

Scene 2: (Ordering)

(Menu on table)
S PTRANS menu to S
(S ask for menu)
S MTRANS signal to W
W PTRANS W to table
S MTRANS 'need menu' to W
W PTRANS W to menu

Lược đồ có cấu trúc – Script



Lược đồ có cấu trúc – Script

Scene 3: (Eating)

C ATRANS F to W
W ATRANS F to S
S INGEST F
(Option: return to scene 2 to order more;
otherwise: goto scene 4)

Scene 4: (exiting)

S MTRANS to W
W ATRANS check to S

W MOVE (write check) ;
W PTRANS W to S
W ATRANS check to S ;
S ATRANS tip to W
S PTRANS S to M;
S ATRANS money to M;
S PTRANS S to out of restaurent.